INTRODUCTION

1. INTRODUCTION

Photography plays a crucial role in capturing and preserving memories at events such as weddings, convocations, college festivals, conferences, and cultural programmes. With the growing volume of digital photographs generated at these events, the development of an efficient and user-friendly system for organizing and distributing them has become essential. This project aims to design and implement SnapTrace, an online platform that uses facial recognition to deliver event photographs to the people who appear in them.

The system is developed using React, Node.js, Express, and MySQL, with face-api, TensorFlow.js, and WebAssembly for facial-recognition processing, ensuring a responsive interface, reliable back-end processing, and secure data management. The primary objective is to provide a centralized platform where administrators, photographers, and attendees can interact seamlessly. Registered users upload a reference selfie, which is converted into a numerical facial descriptor. When event photographs are uploaded, the system detects faces, compares their descriptors with the stored ones, and adds the matching photographs to each user's personal gallery.

The project also simplifies administrative operations by providing authentication, role-based access control, administrative functions, and audit logging within a single system. By integrating these features, SnapTrace reduces manual effort, speeds up photo distribution, and promotes controlled access and responsible handling of biometric information.

Through this project, we demonstrate the practical application of web development, image processing, and database management concepts in addressing real-world challenges in event media management.

1.1 OBJECTIVES The primary objectives of the SnapTrace: Intelligent Event Media Management and Biometric Distribution System are as follows:

- **User-Friendly Platform:**

  To provide an intuitive and user-friendly web interface that enables attendees, photographers, and administrators to access and manage event media according to their respective roles.

- **Automated Photograph Identification:**

  To integrate facial-recognition technology that identifies faces in uploaded event photographs and compares them with registered users' facial descriptors to find potentially relevant images.

- **Efficient Event Media Management:**

To streamline the process of uploading, processing, organizing, and distributing event photographs through an integrated web-based application, reducing the need for manual photograph searching.

- **Personalized Photo Galleries:**

  To provide registered users with galleries containing photographs associated with their accounts through the facial-matching process, making it easier to find relevant event memories.

- **Image Processing and Optimization:**

  To utilize image-processing techniques for validating, resizing, compressing, and generating thumbnails of photographs to support efficient media storage and presentation.

- **Secure User Authentication and Access Control:**

  To implement authentication and role-based authorization mechanisms that restrict access to protected application features according to the roles and permissions assigned to users.

- **Centralized Data Management:**

  To use a MySQL relational database for managing user accounts, settings, image metadata, gallery associations, administrative reports, audit records, and password-reset information.

- **Administrative Management and Monitoring:**

  To provide administrative functionality, reporting, and audit logging to support the management and monitoring of relevant activities within the application.

- **Asynchronous Image Processing:**

  To implement an in-process asynchronous queue that separates photograph-upload responses from subsequent facial-recognition and matching operations.

- **Privacy-Aware Media Distribution:**

  To consider privacy and controlled access when handling reference selfies, facial descriptors, and event photographs, while documenting the security and privacy limitations of the implemented system.

- **Future Scalability and Enhancement:**

  To establish a foundation for future improvements, including distributed job processing, cloud-based media storage, more efficient facial-descriptor searching, stronger authentication, and automated data-retention management.

TECHNOLOGY PROFILE

2. TECHNOLOGY PROFILE

SnapTrace: Intelligent Event Media Management and Biometric Distribution System is developed using a combination of modern web technologies, backend frameworks, relational database systems, image-processing libraries, and computer-vision tools. These technologies work together to provide user authentication, event photograph uploads, facial recognition, image matching, gallery management, and administrative functionality. The selection of these technologies supports the integration of frontend development, backend processing, database operations, and biometric image analysis within a single web-based application.

1. React

React is a JavaScript library used to develop the user interface of SnapTrace. It enables the application to be divided into reusable components, making the interface easier to organize and maintain. React is used to build the frontend pages and interactive elements through which attendees, photographers, and administrators access the system's features.

2. Vite

Vite is a frontend development and build tool used to run and build the React application. It provides a development server and a build process for preparing the frontend for deployment. In SnapTrace, Vite supports the development workflow and the delivery of the frontend application.

3. React Router DOM

React Router DOM is used to manage navigation within the React application. It allows users to move between different application pages without requiring a complete browser-page reload for each navigation action. It supports the frontend routing structure of SnapTrace.

4. HTML, CSS and JavaScript

HTML provides the structural foundation of the web interface, while CSS controls its visual presentation and layout. JavaScript implements interactive behaviour and connects frontend components with backend API services. These technologies work together to create the user interface and support communication with the server.

5. Node.js

Node.js is a JavaScript runtime used to execute the SnapTrace backend. It provides the environment for running server-side application logic, processing API requests, coordinating image-processing tasks, and communicating with the database and computer-vision libraries.

6. Express.js

Express.js is a web application framework for Node.js used to develop backend API endpoints. In SnapTrace, it manages HTTP requests and responses and connects application routes with controllers and supporting services. It also supports middleware-based authentication, authorization, and request processing.

7. MySQL

MySQL is a relational database management system used to store and manage the application's structured data. SnapTrace uses MySQL to maintain user accounts, user settings, image metadata, gallery associations, audit records, reports, and password-reset information. Relational constraints and database operations help maintain relationships between the records used by the application.

8. Face-api

Face-api is a JavaScript library used for face detection, facial-landmark processing, and facial-descriptor generation. In SnapTrace, it supports the computer-vision pipeline that detects faces in reference selfies and uploaded event photographs. The generated descriptors are used for facial comparison and the identification of potentially relevant photographs.

9. TensorFlow.js

TensorFlow.js provides machine-learning functionality in JavaScript environments. SnapTrace uses it as part of the backend computer-vision stack to execute the models required for facial processing. Its integration allows the application to perform the configured inference operations within the Node.js environment.

10. WebAssembly (WASM)

WebAssembly is a portable execution format that can run computational code in supported environments. SnapTrace uses the TensorFlow.js WebAssembly backend for computer-vision inference. This provides a machine-learning execution option within the JavaScript-based backend without requiring a separate GPU-based inference service.

11. Sharp

Sharp is an image-processing library used to prepare and manipulate image files. SnapTrace uses Sharp for image validation and processing operations, including resizing, compression, and thumbnail generation. These operations help prepare uploaded photographs for storage and presentation and support the processing of reference selfies.

12. Multer

Multer is middleware for handling multipart form-data requests, particularly file uploads, in Node.js and Express applications. In SnapTrace, it supports the reception of uploaded reference selfies and event photographs through the backend. Additional validation is used to enforce the application's supported file types and upload-size limits.

13. JSON Web Token (JWT)

JSON Web Token is a token-based mechanism used to represent authentication information between the client and server. SnapTrace uses JWT-based authentication to identify authenticated users when accessing protected backend endpoints. The application also implements token invalidation support for relevant account-security changes.

14. Bcrypt

Bcrypt is a password-hashing library used to protect user passwords. Instead of storing passwords as ordinary readable text, the application stores password hashes that can be verified during authentication. SnapTrace uses this mechanism as part of its account-security implementation.

15. Vitest

Vitest is a JavaScript and TypeScript testing framework used for automated backend tests in SnapTrace. The documented test baseline contains five backend test suites with 44 passing tests. These tests verify selected application behaviour, but they do not establish complete frontend, end-to-end, or biometric-accuracy test coverage.

16. In-Process Asynchronous Queue

SnapTrace uses an in-process JavaScript queue to schedule facial-matching tasks after event photographs are uploaded. This approach allows the upload response to be returned before all matching operations are completed. The queue uses a single worker and is stored in application memory; it is not a distributed or durable job-processing system.

SYSTEM STUDY

3. SYSTEM STUDY

System study is an important phase in the development of any software system. It involves examining the existing methods, identifying their limitations, understanding user requirements, and evaluating possible solutions to determine the most suitable approach. In SnapTrace, the system study focuses on the challenges involved in managing and distributing digital photographs generated during events such as college festivals, convocations, conferences, weddings, and other social or professional gatherings. The study examines the difficulties associated with manually identifying photographs, organizing large image collections, and distributing relevant photographs to individual participants. Based on these observations, an automated, web-based solution using facial-recognition technology is proposed. The system study also considers the functional requirements, user roles, database management, image-processing workflow, access control, and privacy considerations necessary for developing an effective event media management system.

3.1 EXISTING SYSTEM

The existing methods of event photograph management commonly depend on manual organization and conventional photo-sharing platforms. After an event, photographers and event organizers generally upload photographs to shared folders, online albums, websites, or cloud storage platforms. Participants must browse through these collections to find photographs in which they appear. When an event contains hundreds or thousands of photographs, locating a particular individual's images can become a time-consuming and inconvenient process. Manual identification also increases the workload of photographers and event organizers, who may need to sort photographs, create separate collections, or respond to individual requests. Traditional photo-sharing methods generally organize images by folders, filenames, or event categories rather than automatically identifying the people appearing in them. As a result, users may need to examine numerous photographs before finding the images relevant to them. Another limitation is the difficulty of distributing photographs to the appropriate individuals in an organized manner. Sharing a common folder provides convenient access to a large collection, but it does not automatically create personalized galleries for individual participants. It can also be difficult to manage access to photographs when several users, photographers, and administrators are involved. Conventional workflows may also provide limited integration between facial identification, image processing, user authentication, and administrative monitoring. Without an automated identification mechanism, organizing and retrieving relevant photographs becomes increasingly difficult as the size of an event's image collection grows. These limitations highlight the need for an intelligent event media management system that can automate photograph identification, associate relevant images with registered users, and provide personalized galleries through a web-based application.

3.2 PROPOSED SYSTEM

The proposed **SnapTrace — Intelligent Event Media Management & Biometric Distribution System** aims to overcome the limitations of conventional photo-sharing methods by integrating facial-recognition technology with web-based image management. The system provides a platform through which users can register, submit a reference selfie, and access event photographs identified as potentially matching their facial features. SnapTrace is developed using **React** for the frontend, **Node.js and Express.js** for backend services, and **MySQL** for data storage. The facial-recognition pipeline uses **@vladmandic/face-api**, TensorFlow.js, and the WebAssembly backend to detect faces, extract facial landmarks, generate numerical facial descriptors, and compare these descriptors with registered users' descriptors. The Sharp image-processing library is used to resize and optimize uploaded images and generate thumbnails. When a user submits a reference selfie, the system validates and processes the image and generates a 128-dimensional facial descriptor. When photographers upload event photographs, the system stores and processes the images and performs face matching in the background. Photographs identified as matching a registered user's facial descriptor can then be associated with that user and displayed in the user's personalized gallery. The system includes separate access and functionality for users, photographers, and administrators. Authentication and role-based authorization help restrict access to relevant operations. Administrative functions support system management and monitoring, while audit logging records selected administrative activities. The backend uses an asynchronous in-process queue to process uploaded photographs without requiring the upload request to wait for the entire matching operation to finish. SnapTrace aims to reduce the manual effort involved in identifying and distributing event photographs, improve image retrieval, and provide a more organized user experience. However, facial matches are based on a configured similarity threshold and should be understood as automated matching results rather than a guarantee of perfect identification. The system also requires responsible handling of selfies, facial descriptors, and event photographs, with appropriate attention to privacy and access control.

3.3 MODULE DESCRIPTION

SnapTrace is divided into functional modules that work together to support user registration, event photograph processing, facial matching, personalized gallery management, and administration.

1. User Module

Functionalities of User:

- **User Registration:** Allows new users to create accounts by providing necessary information and a reference selfie.
- **Login:** Authenticate themselves to access the application.
- **Password Reset:** Provides a password recovery mechanism through a reset link.
- **Profile Management:** Supports access to and management of available user profile information.
- **Selfie Enrollment:** Allows users to upload a reference selfie for facial-descriptor generation.
- **Personalized Photo Gallery:** Enables users to view event photographs associated with their accounts through the face-matching process.
- **Gallery Management:** Supports access to matched photographs and related gallery information.
- **User Settings:** Allows users to access and update supported application preferences.

2. Photographer Module

Functionalities of Photographer:

- **Login and Authentication:** Provides authenticated access to photographer-related functionality.
- **Event Photograph Upload:** Allows photographers to upload event photographs in supported image formats.
- **Image Validation:** Validates uploaded files against supported formats and configured file-size and upload-count limits.
- **Background Face Matching:** Initiates asynchronous processing to detect faces and compare facial descriptors.
- **Photo Distribution:** Supports associating matched photographs with the relevant registered users.

- **Upload Status and Results:** Provides access to available information about uploaded images and their processing results.

3. Administrator Module

Functionalities of Admin:

- **Administrative Authentication:** Provides access to administrative operations for authorized users.
- **User Management:** Supports viewing and managing user information through available administrative functionality.
- **Photographer Management:** Supports relevant account and system-management operations for photographers.
- **System Monitoring:** Provides access to available administrative information about the application's operation.
- **Audit Log Management:** Records and supports the review of selected administrative activities.
- **Report Management:** Supports the generation and management of reports in the implemented reporting workflow.
- **Access Control:** Uses role-based authorization to restrict administrative operations to permitted users.

4. Facial Recognition and Image Processing Module

Functionalities of the Module:

- **Face Detection:** Detects faces in uploaded selfies and event photographs using the configured face-detection model.
- **Facial Landmark Detection:** Identifies facial landmarks to support the facial-recognition pipeline.
- **Facial Descriptor Generation:** Generates a 128-dimensional numerical descriptor representing detected facial features.
- **Descriptor Comparison:** Compares facial descriptors using Euclidean distance.
- **Match Decision:** Uses a configured distance threshold of 0.50 or lower to identify potential matches.

- **Image Optimization:** Resizes uploaded images and generates thumbnails to support efficient storage and display.
- **Asynchronous Processing:** Processes event photographs through an in-process background queue.

5. Database and Data Management Module

Functionalities of the Module:

- **User Data Management:** Stores and retrieves registered user information.
- **Facial Descriptor Storage:** Stores facial descriptors in the database for subsequent comparison.
- **Photograph Metadata Management:** Maintains image records and associated metadata.
- **Personalized Gallery Mapping:** Maintains associations between users and photographs identified as matches.
- **Settings Management:** Stores user settings when they are explicitly created or updated.
- **Report Metadata Management:** Maintains report-related records in the database.
- **Password Reset Records:** Maintains password-reset tokens and their associated expiry and usage information.

6. Authentication and Security Module

Functionalities of the Module:

- **Password Protection:** Uses password hashing to protect stored account passwords.
- **Token-Based Authentication:** Uses JSON Web Tokens (JWT) to authenticate users.
- **Role-Based Authorization:** Restricts protected operations according to the user's role.
- **Signed Media URLs:** Supports time-limited, HMAC-SHA256-signed URLs for controlled media access.
- **Input Validation:** Validates supported upload formats and configured file restrictions.
- **Administrative Auditing:** Records selected administrative actions to support accountability.

SYSTEM ANALYSIS

4. SYSTEM ANALYSIS

4.1 INTRODUCTION

System analysis is an important phase in the software development process. It involves identifying the problems associated with the existing system, understanding user requirements, examining the operations to be performed, and determining a suitable solution. In the development of SnapTrace, system analysis focuses on the difficulties involved in manually organizing, identifying, and distributing event photographs. The analysis examines how facial-recognition technology can be integrated with a web-based application to simplify photograph identification and provide personalized galleries for registered users. It also considers the requirements for user authentication, image processing, database management, administrative operations, and controlled access to photographs. Based on this analysis, SnapTrace is designed to provide an automated and organized approach to event media management using modern web technologies.

4.2 DEFINITION

System analysis is the systematic study of a proposed or existing system to understand its components, functions, operations, and relationships. It involves breaking down the system into smaller functional units to identify requirements, limitations, and possible improvements. In SnapTrace, system analysis examines the interaction between the frontend, backend services, database, facial-recognition pipeline, and different user roles to determine how the application can efficiently identify and distribute relevant event photographs.

4.3 OBJECTIVES

The main objectives of system analysis for SnapTrace are as follows:

- **Understanding the Existing System:** To study conventional methods of event photograph storage and sharing and identify the difficulties associated with manually finding relevant photographs.
- **Identifying System Requirements:** To determine the functional and technical requirements needed for user registration, selfie enrollment, photograph uploads, facial matching, and personalized gallery management.
- **Developing the Proposed System:** To design a web-based event media management system that uses facial-recognition technology to automate the identification of potentially relevant photographs.
- **Efficient Image Processing:** To analyze and implement the processes required to detect faces, generate facial descriptors, compare them, and associate matching photographs with registered

users.

- **Database Management:** To organize user information, facial descriptors, photograph metadata, gallery associations, and other application records using a relational database.
- **User-Friendly Interface:** To provide an accessible and organized interface through which users, photographers, and administrators can perform their respective operations.
- **Authentication and Access Control:** To incorporate authentication and role-based authorization to restrict access to protected application features.
- **Reducing Manual Effort:** To minimize the time and effort required to identify and distribute photographs compared with conventional manual methods.
- **Privacy and Data Protection:** To consider appropriate handling of selfies, facial descriptors, and event photographs, including access restrictions and responsible data management.
- **System Reliability and Maintainability:** To organize the application into functional components that can be tested, maintained, and improved as requirements evolve.

SYSTEM SPECIFICATION

5. SYSTEM SPECIFICATION

5.1 SOFTWARE SPECIFICATIONS

A Software Requirements Specification (SRS) describes the functional and non-functional requirements of a software system. It provides a detailed overview of the system's expected behaviour, software components, operating environment, and implementation constraints. It helps ensure that the developed application meets the intended requirements and provides a clear understanding of the technologies required for development and execution.

SnapTrace is a web-based Intelligent Event Media Management and Biometric Distribution System developed to automate the identification and distribution of event photographs using facial-recognition technology. The software environment consists of frontend technologies for the user interface, backend technologies for application logic, a relational database for data storage, and computer-vision libraries for facial recognition and image processing.

The software requirements for SnapTrace are as follows:

- **Operating System:** Windows 10 or Windows 11, or a compatible Linux operating system.
- **Frontend:** React 19, Vite 8, React Router DOM 7, HTML, CSS, and JavaScript.
- **Backend:** Node.js 20 or above and Express.js 4.21.2.
- **Database:** MySQL Community Server 8.0.46.
- **Database Connectivity:** MySQL2 with Promise-based support.
- **Facial Recognition:** @vladmandic/face-api 1.7.15.
- **Machine Learning Framework:** TensorFlow.js with the WebAssembly backend.
- **Image Processing:** Sharp for image resizing, optimization, and thumbnail generation.
- **Authentication and Security:** JSON Web Tokens (JWT) and Bcrypt.
- **File Upload Management:** Multer.
- **Email Service:** Nodemailer for password-reset email functionality.
- **Testing Framework:** Vitest for backend testing.
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or another modern browser compatible with the application's frontend.
- **Development Tools:** Visual Studio Code or another suitable code editor, Node.js package management tools, and MySQL administration tools.

These technologies work together to provide a web-based platform for user authentication, selfie enrollment, event photograph uploads, facial matching, personalized galleries, and administrative operations. The React frontend communicates with the Express backend through API endpoints, while MySQL stores user information, image metadata, facial descriptors, gallery associations, and other application records.

5.2 HARDWARE SPECIFICATION

Hardware specifications describe the physical resources required to develop, execute, and test a software application. The required configuration depends on the size of the image collection, the number of users, and the computational workload involved in facial-recognition processing.
SnapTrace performs image processing and facial matching on the backend. Therefore, adequate processor performance, memory, and storage capacity are important for handling uploaded photographs and processing facial descriptors. The following configuration provides a practical guideline for development and testing.

Minimum Hardware Requirements

- **Processor:** Dual-core processor or equivalent.
- **Memory (RAM):** 4 GB.
- **Storage:** At least 10 GB of available disk space for the application, dependencies, database, and test images.
- **Display:** Standard PC monitor or laptop display.
- **Input Devices:** Keyboard and mouse or compatible touchpad.
- **Network:** Internet or local network connectivity for communication between the frontend, backend, and database when deployed separately.

Recommended Hardware Requirements

- **Processor:** Quad-core Intel or AMD processor or equivalent.
- **Memory (RAM):** 8 GB to 16 GB.
- **Storage:** SSD with sufficient free space for application files, database records, uploaded photographs, thumbnails, and reports.
- **Display:** Standard HD display or higher.
- **Input Devices:** Keyboard and mouse or compatible touchpad.
- **Network:** Stable broadband or local network connection.

6. FEASIBILITY ANALYSIS

The feasibility analysis for **SnapTrace — Intelligent Event Media Management & Biometric Distribution System** evaluates the practicality of developing and implementing the project in terms of technical, economic, operational, schedule, and legal feasibility. This analysis helps determine whether the proposed system can be developed using the available technologies, resources, and time while meeting its intended objectives. SnapTrace aims to simplify event photograph management by using facial-recognition technology to identify potentially relevant photographs and associate them with registered users. The feasibility study examines the suitability of the technology stack, development costs, usability, project schedule, and considerations related to the handling of personal and biometric information.

6.1 TECHNICAL FEASIBILITY

SnapTrace is technically feasible because it uses established web development technologies and machine-learning libraries. The frontend is developed using React, while Node.js and Express.js provide backend services. MySQL is used for structured data storage, and @vladmandic/face-api, TensorFlow.js, and the WebAssembly backend support facial detection and descriptor comparison. The Sharp library is used for image resizing, optimization, and thumbnail generation. These technologies support the implementation of user registration, authentication, selfie enrollment, event photograph uploads, facial matching, and personalized photo galleries. The system uses a modular architecture that separates the frontend, backend, database, and image-processing operations. This organization supports testing and maintenance. However, facial-recognition performance depends on factors such as image quality, lighting, face orientation, and the number of photographs being processed. The current system uses an in-process asynchronous queue rather than a distributed job-processing service. Therefore, the implementation is suitable for its intended development and small-to-medium event use cases, while larger deployments may require additional infrastructure and performance evaluation.

6.2 ECONOMIC FEASIBILITY

SnapTrace is economically feasible because its principal development technologies are available as open-source software or through freely available development tools. React, Node.js, Express.js, MySQL Community Server, and the facial-recognition libraries help reduce software licensing costs. The application can be developed and tested on an existing computer without requiring dedicated commercial machine-learning hardware. The main costs associated with the project may include computer hardware, internet connectivity, database and media storage, deployment infrastructure, domain registration, and maintenance. Additional expenses may arise if the system is deployed on a paid hosting platform or requires increased storage and processing capacity. Since the project uses existing software libraries and a web-based architecture, the initial development cost can be kept relatively low. The expected benefits include reduced manual effort in locating photographs, more organized

image distribution, and improved convenience for event participants. Actual cost savings will depend on the scale of deployment and the resources required to operate the system.

6.3 OPERATIONAL FEASIBILITY

SnapTrace is operationally feasible because it provides separate functionality for registered users, photographers, and administrators. Users can register, submit a reference selfie, and view photographs associated with their accounts. Photographers can upload event photographs for processing, while administrators can access the available management, reporting, and audit-related functions. The web-based interface allows users to access the system through a compatible browser without requiring a separate desktop application. Automated face detection and descriptor comparison reduce the need to manually inspect every photograph to identify potential matches. Background processing also allows the upload request to finish before the complete matching workflow has finished. The system nevertheless requires users to provide suitable reference selfies and photographers to upload supported image files. Facial matching results may not always be accurate, so the system should not be treated as guaranteeing correct identification. Appropriate access controls and responsible handling of selfies, facial descriptors, and event photographs are also necessary for practical use.

6.4 SCHEDULE FEASIBILITY

SnapTrace is schedule-feasible when developed through a structured, incremental approach. The project can be divided into stages such as requirement analysis, frontend development, backend and database implementation, facial-recognition integration, image-processing implementation, and testing. The modular structure allows individual components to be developed and verified before they are integrated into the complete application. Existing frameworks and libraries also reduce the need to implement common functions, such as routing, database connectivity, authentication, and image processing, entirely from scratch. The project includes backend unit tests to verify selected application functions. However, the facial-recognition workflow and background-processing queue are not comprehensively covered by those unit tests, and frontend end-to-end testing has not been established. Additional integration and performance testing would therefore be required before a larger production deployment. The overall schedule feasibility depends on the project deadline, available development time, and the effort needed for verification and refinement.

6.5 LEGAL FEASIBILITY

SnapTrace processes personal information, including account details, reference selfies, facial descriptors, and event photographs. Therefore, privacy, security, and responsible data handling are important considerations in its development and deployment.

The system incorporates certain protective measures, including password hashing, token-based authentication, role-based authorization, and controlled media-access mechanisms. These features help reduce the risk of unauthorized access to protected application functions. The system also supports account-related data management and records selected administrative activities.

However, the presence of these technical controls does not establish legal compliance or certification. The implementation has limitations that require attention, including the need for stronger controls over direct access to uploaded media, a secure and consistent configuration of signing secrets, and a defined data-retention and deletion process. Clear user consent, appropriate privacy notices, and procedures for handling personal data should also be addressed before deployment.

The project should be evaluated against applicable Indian data-protection requirements, including the Digital Personal Data Protection Act, 2023, and relevant rules and commencement provisions applicable at the time of deployment. Consequently, SnapTrace is technically designed with certain privacy and security considerations, but its legal compliance must be assessed separately before it is used in a real-world environment.

SYSTEM DESIGN

7. SYSTEM DESIGN

System design is an important phase in software development that translates the requirements identified during system analysis into a structured software solution. It defines the architecture, components, data flow, input and output processes, database structure, and interactions between different parts of the application. The purpose of system design is to establish a clear plan for implementing a system that meets the specified requirements while remaining maintainable and reliable.

In SnapTrace, the system design focuses on developing an intelligent event media management and biometric distribution system that automates the identification and distribution of event photographs. The design integrates a React-based frontend, a Node.js and Express backend, a MySQL database, and a facial-recognition pipeline. It defines how users register and submit reference selfies, how photographers upload event photographs, how facial descriptors are generated and compared, and how matched photographs are associated with individual user galleries.

The design is divided into logical and physical aspects. Logical design describes the major components, their relationships, the information flowing through the system, and the data stored in the database. Physical design describes the implementation using the selected programming languages, frameworks, libraries, database tables, and application services.

7.1 ARCHITECTURAL DESIGN

Software architectural design defines the major components of a system and the relationships between them. It provides a structural framework for understanding how the application operates and how its components communicate with one another.

SnapTrace follows a **three-tier web application architecture** consisting of the presentation layer, application layer, and data layer.

7.1.1. Presentation Layer

The presentation layer is developed using React 19, Vite 8, React Router DOM 7, HTML, CSS, and JavaScript. It provides the user interface through which users, photographers, and administrators interact with the application. It supports account-related operations, selfie submission, photograph uploads, gallery access, and administrative functions.

7.1.2. Application Layer

The application layer is implemented using Node.js and Express.js. It handles API requests, authentication, role-based authorization, input validation, file uploads, image-processing operations, facial recognition, and database communication. The application uses an asynchronous in-process queue to process uploaded photographs in the background.

7.1.3. Data Layer

The data layer uses MySQL Community Server to store structured application information. The database maintains user records, user settings, image metadata, user-gallery associations, administrative audit records, reports, and password-reset tokens. Facial descriptors are stored as JSON data associated with user records. The facial-recognition pipeline operates within the backend processing workflow. It uses @vladmandic/face-api, TensorFlow.js, and the WebAssembly backend to detect faces, identify facial landmarks, generate 128-dimensional facial descriptors, and compare descriptors using Euclidean distance.

The overall architecture can be represented as:

User / Photographer / Administrator → React Frontend → Express REST API → Application Services → MySQL Database

The facial-recognition and image-processing services are invoked by the backend as required. Uploaded photographs are processed asynchronously, and potential matches are associated with the relevant users.

7.2 INPUT DESIGN

Input design specifies how information enters the system, how it is validated, and how it is processed. A well-designed input mechanism helps reduce invalid data, prevents avoidable processing errors, and provides users with a consistent interaction experience.

In SnapTrace, input design covers account information, reference selfies, event photographs, user settings, and administrative requests. The application validates inputs before performing the corresponding operations.

The major inputs are described below.

1. User Registration Input

Users provide the information required to create an account. The backend validates the submitted data before storing the account information in the database. Passwords are hashed using Bcrypt before storage.

2. Login Input

Users submit their login credentials through the authentication interface. The backend verifies the credentials and issues a JSON Web Token (JWT) when authentication succeeds. Protected operations require appropriate authentication and authorization.

3. Reference Selfie Input

A registered user can upload a reference selfie for facial enrollment. The accepted image formats are JPEG and PNG, with a maximum file size of 5 MB. The image is processed using Sharp, with

its maximum dimension reduced to 600 pixels. The facial-recognition pipeline requires exactly one detected face for enrollment. A 128-dimensional facial descriptor is generated and stored in the database.

4. Event Photograph Input

Photographers can upload event photographs for processing. The system accepts JPEG and PNG images, with a maximum size of 30 MB per file and a maximum of 10 files per request. Sharp is used to resize and optimize the images and generate thumbnails.

5. User Settings Input

Users can update the application preferences supported by the system. The backend validates and stores the settings in the database when they are explicitly created or updated.

6. Administrative Input

Authorized administrators submit requests for available administrative functions, including user management, report-related operations, and other supported management activities. Role-based authorization restricts access to protected operations.
Input validation and appropriate error responses help prevent unsupported file formats, oversized uploads, invalid authentication requests, and other avoidable processing problems.

7.3 OUTPUT DESIGN

Output design describes how processed information is presented to users. The output should be clear, organized, and relevant to the user's role and activities. SnapTrace provides application outputs through its web interface and supporting API responses.
The main outputs of the system include the following:

1. Authentication Output

The system provides authentication results and appropriate responses for successful or unsuccessful login and password-reset operations.

2. Personalized Photo Gallery

Users can view photographs associated with their accounts through the personalized gallery. These associations are generated through the facial-matching workflow.

3. Photograph Upload Response

After uploaded files have been accepted and stored, the backend returns an HTTP 201 response before background facial matching is necessarily complete. Therefore, successful upload acceptance does not mean that every photograph has already been processed or matched.

4. Facial-Matching Output

The matching process compares facial descriptors using Euclidean distance. A distance of 0.50 or lower is treated as a potential match according to the configured threshold. The matching workflow produces associations between photographs and registered users when the configured criteria are satisfied.

5. Image and Thumbnail Output

The system generates optimized image files and thumbnails to support image display and retrieval.

6. Administrative Output

Authorized administrators can access available management information, selected audit records, and report-related outputs.

7. Report Output

The reporting workflow generates JSON report files in the backend uploads directory and maintains associated metadata in the database.

The output design aims to provide an organized way to access photographs and administrative information. The accuracy of facial matching depends on image quality and other factors, so matching results should not be interpreted as a guarantee of correct identification.

7.4 DATABASE DESIGN

Database design defines how information is organized, stored, related, and retrieved. SnapTrace uses MySQL Community Server with the InnoDB storage engine and the utf8mb4_unicode_ci character collation. The database supports the storage and management of account information, facial descriptors, image records, gallery associations, and administrative data.
The database consists of seven live physical tables.

1. users

This table stores registered user information and account-related data. It also contains the face_embedding field, which stores the user's 128-dimensional facial descriptor as JSON data. The user's updated_at field reflects updates to the user record.

2. user_settings

This table stores user preferences when a settings record has been explicitly created or updated. The application can return default settings when no corresponding record exists.

3. images

This table stores image-related information and metadata required by the application for managing uploaded photographs.

4. user_gallery_photos

This table maintains the association between users and photographs identified as potential matches. It supports a many-to-many relationship between users and images and stores match-related information, including confidence values. A unique constraint on (user_id, photo_id) prevents duplicate associations for the same user and photograph.

5. admin_audit_log

This table records selected administrative activities to support accountability and operational review.

6. reports

This table stores metadata associated with generated reports. The corresponding report contents are written to JSON files.

7. password_reset_tokens

This table stores password-reset token records, including information related to token usage and expiry.

Primary Key:

A primary key uniquely identifies a record in a table. It helps maintain the identity of individual database records.

Foreign Key:

A foreign key establishes a relationship between records in different tables and helps maintain referential integrity. SnapTrace uses relational associations to connect users, images, and gallery records.

Normalization:

Database normalization organizes data to reduce unnecessary duplication and improve consistency. SnapTrace uses relational tables and defined relationships to structure its application data. The database design should be evaluated against normalization principles and the specific constraints of each table.

Database Relationships

The main database relationships include:

- User records are associated with user settings where a settings record exists.
- User records are associated with facial descriptors used during matching.
- Image records are associated with users through the user_gallery_photos mapping table.
- Report records maintain metadata for generated report files.
- Password-reset records are associated with accounts and their reset operations.

The user_gallery_photos table is present in the live database and schema dump. However, the available migration files do not fully reproduce the live schema, so the migration set requires correction before it can be relied upon to recreate the complete database independently.

7.5 MODULARIZATION

Modularization is the process of dividing a software system into smaller functional units. Each module is responsible for a specific group of related operations. This approach improves code organization, simplifies testing, and makes maintenance easier because changes can often be made within a particular component without affecting the entire application.
SnapTrace is organized into the following functional modules:

- **Authentication Module:** Handles account authentication, JWT generation and validation, password hashing, and password-reset operations.
- **User Management Module:** Handles user records, profile-related operations, and supported user settings.
- **Selfie Enrollment Module:** Accepts reference selfies, validates uploaded files, and generates facial descriptors.
- **Photograph Upload Module:** Accepts event photographs, validates files, and stores the uploaded images.
- **Image Processing Module:** Resizes images, optimizes image files, and generates thumbnails using Sharp.
- **Facial Recognition Module:** Detects faces, identifies facial landmarks, generates descriptors, and compares facial descriptors.
- **Gallery Management Module:** Maintains associations between users and photographs identified as potential matches.
- **Administrative Module:** Supports available administrative functions and records selected administrative activities.
- **Reporting Module:** Generates JSON reports and maintains their metadata.
- **Database Access Module:** Provides database operations for retrieving and modifying application data.

The modules communicate through defined application interfaces and backend services. This structure makes the application easier to understand and provides a basis for future improvements.

7.6 CODING PHASE

The coding phase translates the system design into executable software. During this phase, the frontend, backend, database operations, authentication mechanisms, and facial-recognition functionality are implemented using the selected technologies.

The SnapTrace frontend is developed using React and JavaScript, with Vite supporting the development and build process. React Router DOM manages frontend navigation. The interface communicates with the backend through REST API endpoints.

The backend uses Node.js and Express.js to implement API routes, request handling, authentication, authorization, file uploads, and application logic. The mysql2/promise library provides Promise-based database connectivity. Bcrypt is used for password hashing, and JWT is used for token-based authentication.

The facial-recognition workflow uses @vladmandic/face-api and TensorFlow.js with the WebAssembly backend. Sharp performs image-processing operations, while Multer manages multipart file uploads. An in-process asynchronous queue coordinates background processing of event photographs.

The coding process also includes input validation, error handling, database operations, and testing of implemented functions. The backend is tested using Vitest. The existing automated tests verify selected backend behaviour, but comprehensive browser end-to-end testing and full evaluation of facial-recognition accuracy remain outside the demonstrated test coverage.

7.7 CODING DESIGN

Coding design establishes conventions that improve readability, reliability, and maintainability. A well-organized codebase uses meaningful names, separates responsibilities, validates inputs, and handles errors consistently.

SnapTrace uses a component-based frontend structure and a backend organized around API routes and application logic. The frontend components are responsible for presenting information and collecting user input, while the backend handles protected operations, data processing, and database interaction. The following principles are relevant to the coding design:

- **Meaningful Naming:** Variables, functions, and components should have names that clearly communicate their purpose.
- **Separation of Responsibilities:** User-interface logic, API handling, facial recognition, image processing, and database operations should remain logically separated.
- **Input Validation:** User-provided values and uploaded files should be validated before processing.
- **Error Handling:** Expected errors should produce appropriate responses instead of causing uncontrolled application failures.

- **Security Practices:** Password hashing, authentication checks, role-based authorization, and parameterized SQL queries help reduce common security risks.
- **Code Reusability:** Shared components and functions can be reused to avoid unnecessary duplication.
- **Testing:** Automated tests help verify implemented behaviour and reduce regressions during maintenance.
- **Configuration Management:** Environment-specific secrets and configuration values should be managed carefully rather than relying on insecure fallback values.

These principles support the maintainability of the project. Additional security review and broader testing are necessary before treating the application as production-ready.

7.8 DESIGN METHODOLOGY

The design methodology used in SnapTrace follows an iterative approach in which the application is developed and refined through related implementation stages. The project can be understood through requirements analysis, architectural design, database design, frontend and backend implementation, facial-recognition integration, and verification.

During logical design, the system's users, inputs, outputs, functional modules, database relationships, and information flow are identified. This stage establishes how the application should respond to user actions and how its components should interact.

During physical design, the logical structure is implemented using React, Node.js, Express.js, MySQL, and the selected image-processing and facial-recognition libraries. API endpoints, database tables, upload handling, authentication, and matching operations are implemented as working software components.

The implementation is then verified through backend tests and functional checks. This iterative approach makes it possible to identify issues, refine individual components, and integrate improvements. It should be understood as a practical development approach rather than a claim that a formally audited development methodology was followed.

7.8.1 LOGICAL DESIGN

Logical design describes the system's major functions, data flow, inputs, outputs, and relationships without concentrating on low-level implementation details.

In SnapTrace, the logical design begins when a user registers and submits a reference selfie. The application validates the selfie and generates a facial descriptor. The descriptor is stored with the user's account information.

When a photographer uploads event photographs, the backend validates and stores the images and initiates background processing. The facial-recognition pipeline detects faces, generates descriptors, and compares them with registered user descriptors. When the configured matching threshold is

satisfied, the system associates the relevant photograph with the user through the gallery mapping table.

Users can then access the photographs associated with their accounts. Administrators can use the available management and reporting functionality to perform authorized operations.

The logical design includes the following major elements:

- **Inputs:** Account details, login credentials, reference selfies, event photographs, settings, and administrative requests.
- **Processing:** Authentication, input validation, image optimization, face detection, descriptor generation, descriptor comparison, and gallery association.
- **Data Storage:** User information, facial descriptors, image metadata, gallery associations, settings, audit records, reports, and password-reset records.
- **Outputs:** Authentication responses, personalized galleries, upload responses, report files, and administrative information.

7.8.2 PHYSICAL DESIGN

Physical design describes how the logical design is implemented using specific technologies, software components, database structures, and deployment resources.

SnapTrace implements the presentation layer using React 19 and Vite 8. The backend is implemented using Node.js and Express.js, and MySQL stores structured application data. The facial-recognition workflow uses @vladmandic/face-api and TensorFlow.js with the WebAssembly backend, while Sharp handles image resizing and thumbnail generation.

Uploaded files are processed according to the configured file-format, file-size, and image-processing restrictions. The backend uses JWT-based authentication and role-based authorization to protect supported operations. MySQL tables and relational constraints maintain the associations required for user galleries and application records.

The asynchronous image-processing queue is maintained in application memory. This allows processing to occur after the upload response, but queued jobs are not stored in a durable distributed queue. A server restart can therefore affect pending work. This is an implementation limitation to consider when planning a larger deployment.

7.8.3 MODULAR DESIGN

Modular design divides the application into components with clearly defined responsibilities. This reduces complexity and supports independent development, testing, and maintenance.
SnapTrace uses separate frontend components and backend functionality for authentication, user management, image uploads, facial recognition, image processing, gallery associations, administration, reporting, and database access.
The modular design provides several advantages:

- **Simplified Development:** Individual features can be implemented and examined separately.
- **Easier Testing:** Components can be tested independently where appropriate.
- **Improved Maintenance:** Changes to one functional area can often be made without redesigning the complete application.
- **Code Reusability:** Common functions can be reused across different parts of the system.
- **Better Organization:** The separation of frontend, backend, and database responsibilities makes the application easier to understand.
- **Future Enhancement:** Individual components can be extended as new requirements arise.

DATA FLOW DIAGRAM

8. DATA FLOW DIAGRAMS

8.1 INTRODUCTION TO DFD

A DFD also known as 'bubble chart' has the purpose of clarifying system requirements and identifying major transformations. It shows the flow of data through a system. It is a graphical tool because it presents a picture. The DFD may be partitioned into levels that represent increasing information flow and functional detail. Four simple notations are used to complete a DFD. These notations are given below :-

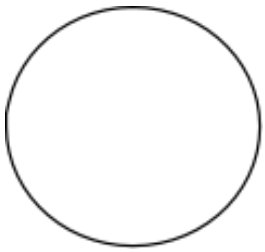
DATA FLOW

The data flow is used to describe the movement of information from one part of the system to another part. Flows represent data in motion. It is a pipe line through which information flows. Data flow is represented by an arrow.



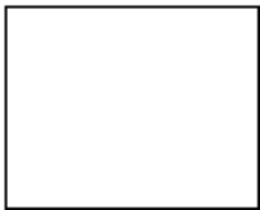
PROCESS

A circle or bubble represents a process that transforms incoming data to outgoing data. Process shows a part of the system that transforms inputs to outputs.



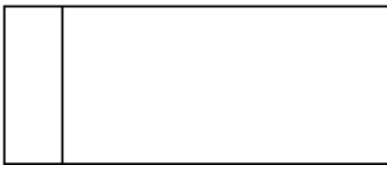
EXTERNAL ENTITY

A square defines a source or destination of system data. External entities represent any entity that supplies or receive information from the system but is not a part of the system.



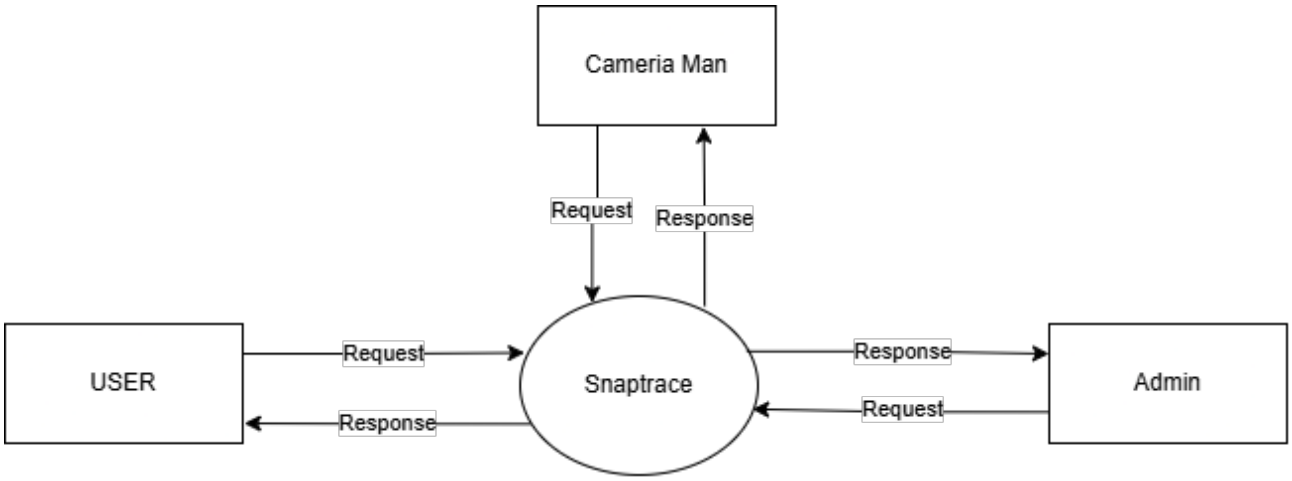
DATA STORE

The data store represents a logical file. A logical file can represent either a data store symbol which can represent either a data structure or a physical file on disk. The data store is used to collect data at rest or a temporary repository of data. It is represented by open rectangle.

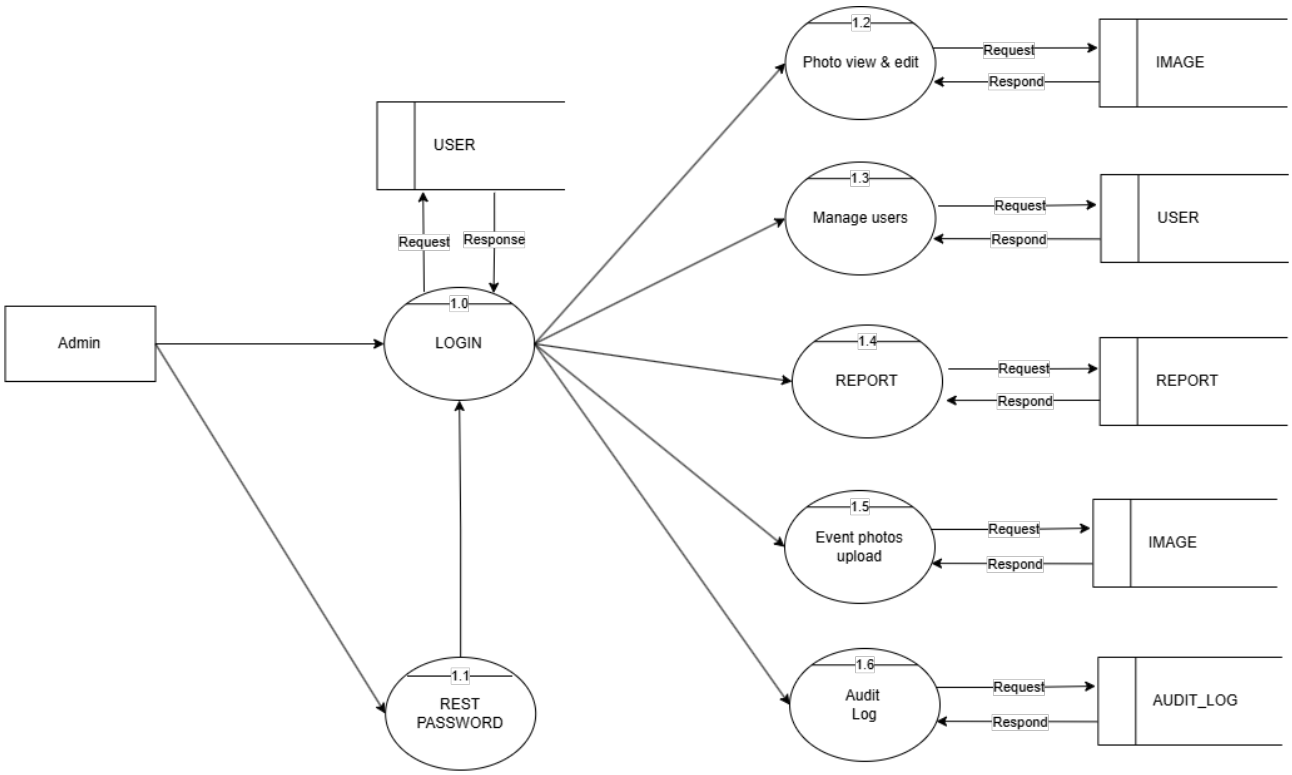


8.2 DATA FLOW DIAGRAMS

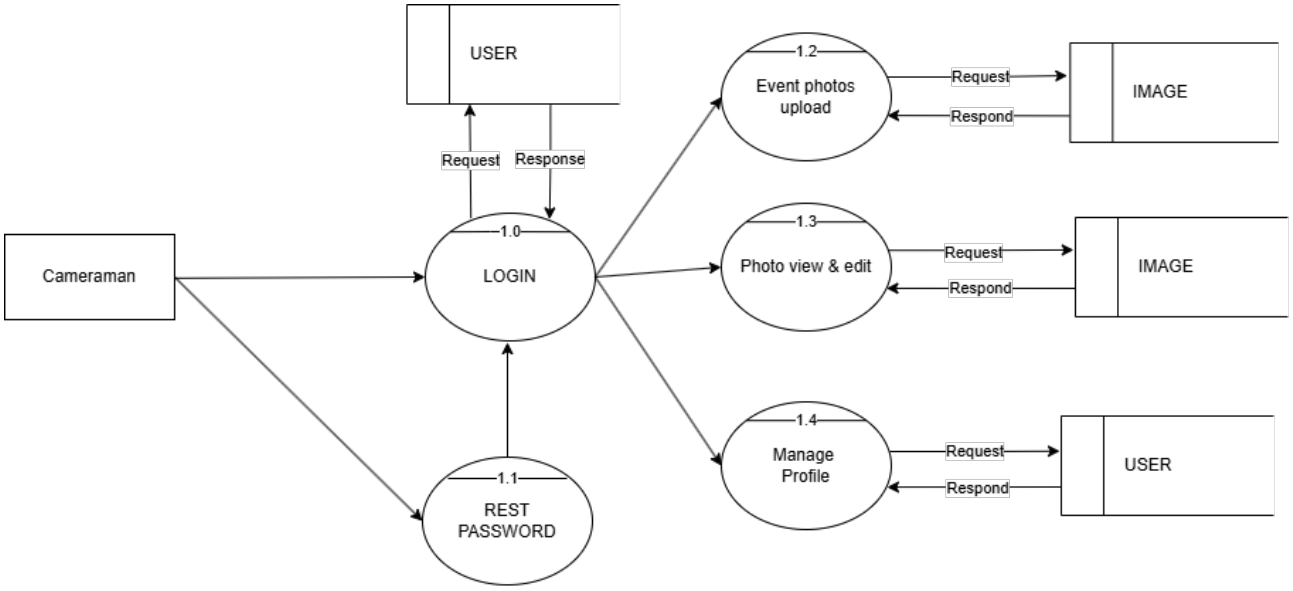
a). DFD Level 0



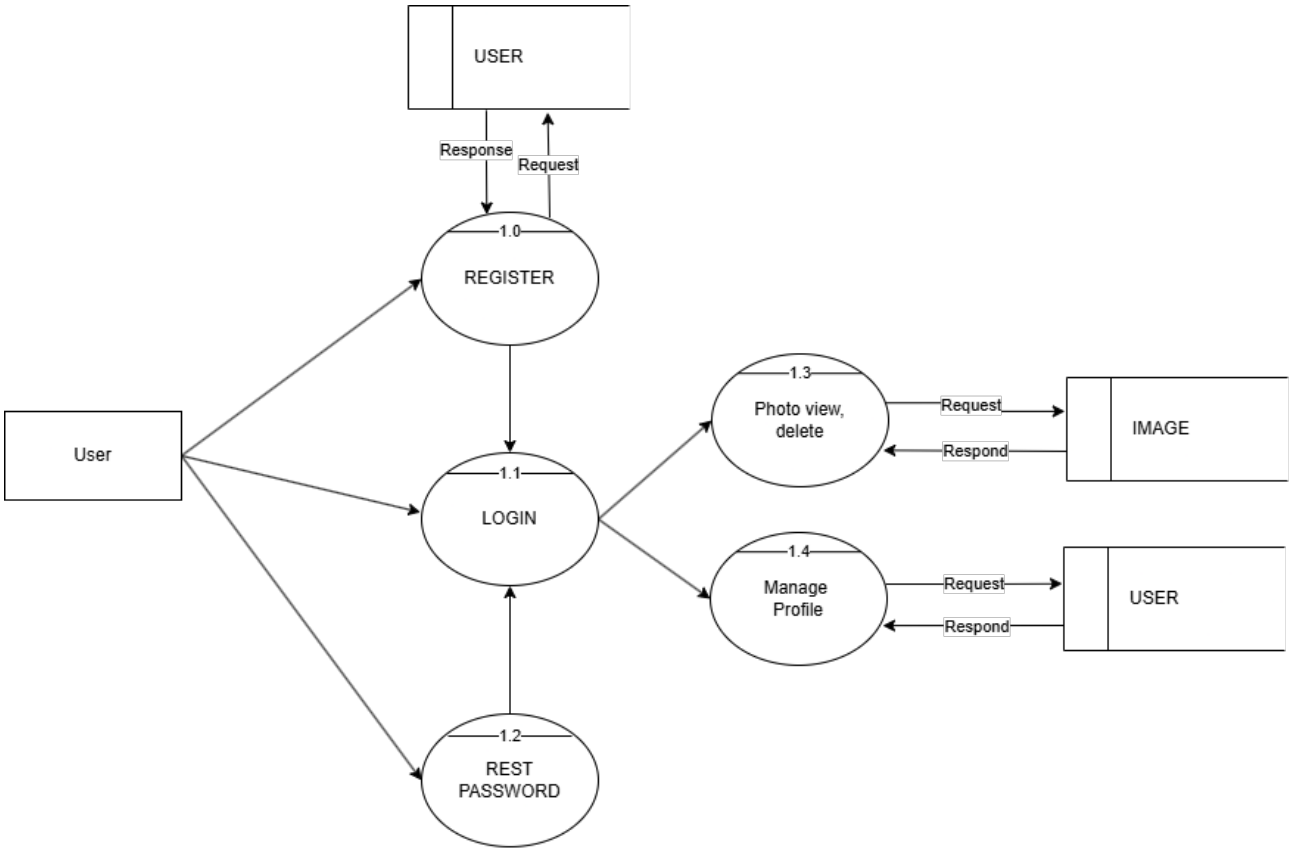
b). DFD Level 1 Admin



c). DFD Level 1 Cameraman



d). DFD Level 1 User



WEB APPLICATION DEVELOPMENT AND DATABASE MANAGEMENT

9. WEB APPLICATION DEVELOPMENT AND DATABASE MANAGEMENT

WEB APPLICATION DEVELOPMENT

Web application development is the process of designing, developing, testing, and maintaining applications that users can access through a web browser over a network. Unlike traditional desktop applications, web applications generally do not require users to install the complete application on their devices. Instead, users can access the application through browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.

SnapTrace is a web-based application developed to simplify the management and distribution of event photographs using facial-recognition technology. It provides different functionalities for registered users, photographers, and administrators. Users can register, submit a reference selfie, and view photographs associated with their accounts. Photographers can upload event photographs for processing, while administrators can access available management and reporting functions.

Web application development can be divided into two major areas: frontend development and backend development.

Frontend Development

Frontend development involves creating the user interface through which users interact with the application. It uses technologies such as HTML, CSS, and JavaScript to structure, style, and implement interactive web pages.

SnapTrace uses React 19 to develop its user interface, Vite 8 as the development and build tool, and React Router DOM 7 for client-side navigation. CSS is used to style the interface, while JavaScript handles application interactions and frontend logic. The frontend communicates with the backend through API requests to perform operations such as authentication, selfie submission, photograph uploads, and gallery retrieval.

Backend Development

Backend development handles application logic, request processing, authentication, database communication, and other operations that occur on the server. It receives requests from the frontend, processes the submitted information, interacts with the database, and returns appropriate responses.

SnapTrace uses Node.js as its server-side JavaScript runtime and Express.js to implement REST API endpoints. The backend manages user authentication, role-based authorization, file uploads, image processing, facial recognition, gallery associations, and reporting operations.

The facial-recognition workflow uses @vladmandic/face-api, TensorFlow.js, and the WebAssembly backend. It detects faces in images, generates numerical facial descriptors, and compares them to identify potentially relevant photographs. The Sharp library is used to resize uploaded images, optimize image files, and generate thumbnails. Photograph matching is performed through an asyn-

chronous in-process queue so that the upload response does not need to wait for the entire matching process to finish.

Web applications commonly use databases to store persistent information such as user accounts, application settings, uploaded image metadata, and other records. In SnapTrace, the frontend, backend, and database work together to provide a centralized platform for event photograph management.

WHAT IS A DATABASE? WHY IS IT IMPORTANT FOR WEB DEVELOPMENT?

Data refers to information that can be collected, stored, processed, and retrieved, such as text, numerical values, account information, and image metadata. A database is an organized collection of data that allows efficient storage and retrieval, and a Database Management System (DBMS) is the software used to create, manage, and access it.

Databases are important in web applications because information such as account details and settings must persist between user sessions and across server restarts. Database systems are commonly classified as relational, document-oriented, key-value, graph, or object-oriented. A relational database organizes data into tables of rows and columns, connected through keys and constraints.

Structured Query Language (SQL) is used with relational databases to perform CRUD operations: Create, Read, Update, and Delete. Relational systems also support constraints and transactions that help maintain data consistency.

SnapTrace uses MySQL Community Server 8.0.46 to store user records, settings, image metadata, user–photograph associations, administrative audit records, report metadata, and password-reset tokens. The users table also stores facial descriptors as JSON data for facial matching.

The database supports authentication and personalized content delivery. After a user logs in, the backend verifies the credentials and uses the authenticated identity to control access to protected operations. When facial matching finds a potential match, the association between the user and the photograph is recorded, and the personalized gallery uses these associations to retrieve the user's photographs.

Primary keys identify individual records, while foreign keys establish relationships and maintain referential integrity. The user_gallery_photos table maps users to images, so one user can be linked to many photographs and one photograph to many users. The Node.js backend accesses MySQL through the mysql2/promise library, and parameterized queries help reduce the risk of SQL injection.

In conclusion, web development and database management are essential to SnapTrace. React provides the interactive frontend, Node.js and Express.js handle server-side operations, and MySQL maintains persistent data. Together with facial recognition and image processing, they let SnapTrace organize event photographs, match them to registered users, and provide personalized gallery access.

TABLES

10. TABLES

a). Admin Audit Log

FIELD	TYPE	CONSTRAINT
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
admin_id	INT	NOT NULL (no declared FK in migration)
action	VARCHAR(50)	NOT NULL
target_type	VARCHAR(30)	NOT NULL
target_id	INT	DEFAULT NULL
ip_address	VARCHAR(45)	DEFAULT NULL
user_agent	VARCHAR(500)	DEFAULT NULL
success	TINYINT(1)	NOT NULL, DEFAULT 1
metadata	JSON	DEFAULT NULL
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

b). Password Token

FIELD	TYPE	CONSTRAINT
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
user_id	INT UNSIGNED	NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE (signedness mismatch documented)
token	VARCHAR(255)	UNIQUE, NOT NULL
expires_at	DATETIME	NOT NULL
used	TINYINT(1)	NOT NULL, DEFAULT 0
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

c). Image

FIELD	TYPE	CONSTRAINT
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
filename	VARCHAR(255)	NOT NULL
filepath	VARCHAR(500)	NOT NULL
original_name	VARCHAR(255)	DEFAULT NULL
mime_type	VARCHAR(100)	DEFAULT NULL
file_size	BIGINT UNSIGNED	DEFAULT 0
uploaded_by	INT	DEFAULT NULL
uploaded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
thumbnail_path	VARCHAR(500)	NOT NULL, DEFAULT ”
status	ENUM(’pending’, ’processing’, ’completed’, ’failed’)	NOT NULL, DEFAULT ’completed’

d). User Gallery

FIELD	TYPE	CONSTRAINT
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
user_id	INT	NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE
photo_id	INT	NOT NULL, FOREIGN KEY → images.id, ON DELETE CASCADE
match_confidence	FLOAT	NOT NULL, DEFAULT 0
matched_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

e). Reports

FIELD	TYPE	CONSTRAINT
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
name	VARCHAR(255)	NOT NULL
type	VARCHAR(50)	NOT NULL
status	ENUM('queued', 'generating', 'completed', 'failed')	NOT NULL, DEFAULT 'queued'
generated_by	INT	NOT NULL (no declared FK in migration)
file_path	VARCHAR(500)	DEFAULT NULL
metadata	JSON	DEFAULT NULL
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
completed_at	DATETIME	DEFAULT NULL

f). User Settings

FIELD	TYPE	CONSTRAINT
user_id	INT	PRIMARY KEY, FOREIGN KEY → users.id, ON DELETE CASCADE
viewed_photos	TINYINT(1)	NOT NULL, DEFAULT 1
email_photos	TINYINT(1)	NOT NULL, DEFAULT 1
keep_face	TINYINT(1)	NOT NULL, DEFAULT 0
email_notifs	TINYINT(1)	NOT NULL, DEFAULT 1
sms_notifs	TINYINT(1)	NOT NULL, DEFAULT 0
marketing	TINYINT(1)	NOT NULL, DEFAULT 0

g). User

FIELD	TYPE	CONSTRAINT
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, DEFAULT 'user'
status	ENUM('active', 'suspended', 'inactive')	NOT NULL, DEFAULT 'active'
token_valid_after	DATETIME	DEFAULT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
last_login_at	DATETIME	DEFAULT NULL
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP
phone	VARCHAR(30)	DEFAULT NULL
avatar_url	VARCHAR(500)	DEFAULT NULL
face_scan_url	VARCHAR(500)	DEFAULT NULL

SYSTEM TESTING

11. SYSTEM TESTING

System testing is an important phase of software development and quality assurance. It involves evaluating the behaviour of a software application to determine whether it satisfies the specified requirements. Testing helps identify errors, verify the correctness of individual modules, and examine the interaction between different components of the system.

SnapTrace is an Intelligent Event Media Management and Biometric Distribution System developed to simplify the identification and distribution of event photographs using facial-recognition technology. Testing of SnapTrace focuses on user authentication, reference selfie enrollment, event photograph uploads, image processing, facial matching, personalized galleries, database operations, and administrative functions.

The backend application uses Vitest for automated testing. The recorded test execution includes five backend test suites containing 44 tests, all of which passed. However, these results represent only the behaviours covered by those tests. Comprehensive frontend end-to-end testing, formal user acceptance testing, performance benchmarking, and biometric accuracy evaluation have not been established by the available test evidence.

11.1 TESTING TECHNIQUES

Software testing can be performed at different levels to evaluate the correctness and reliability of an application. The following testing techniques are relevant to SnapTrace:

- Unit Testing
- Integration Testing
- Validation Testing
- System Testing
- Output Testing
- User Acceptance Testing

Each technique focuses on a different aspect of the application. The following sections describe how these techniques apply to SnapTrace and distinguish verified results from testing activities that require further execution.

11.1.1 UNIT TESTING

Unit testing is the process of testing individual functions, components, or modules to verify that they produce the expected results. It is generally performed during development to identify errors before the modules are integrated into the complete application.
SnapTrace uses Vitest as its backend testing framework. Automated tests are used to verify selected backend operations and application logic. The reported test execution contains five backend test suites with 44 tests.

Recorded Test Result:

- Testing framework: Vitest
- Number of test suites: 5
- Total tests: 44
- Passed tests: 44
- Failed tests: 0

The results indicate that all 44 tests passed during the recorded execution. This provides evidence that the behaviours covered by those tests worked as expected in that run. However, it does not establish complete test coverage of every module. The facial-recognition inference pipeline and asynchronous image-processing queue are not comprehensively covered by the available unit-test evidence.

11.1.2 INTEGRATION TESTING

Integration testing examines whether different modules communicate correctly and exchange information as expected. It focuses on identifying problems at the interfaces between application components.
SnapTrace integrates a React frontend with a Node.js and Express backend through REST API endpoints. The backend communicates with the MySQL database using the mysql2/promise library. It also integrates file-upload handling, image processing, facial-recognition services, and gallery-management operations.
Integration testing should verify the following interactions:

- Communication between the frontend and backend API.
- Database insertion, retrieval, and updating of application records.
- Uploading and storing reference selfies and event photographs.
- Processing photographs and generating facial descriptors.

- Creating associations between users and potentially matching photographs.
- Retrieving the correct photographs for a user's personalized gallery.

These checks are important for ensuring that individual modules work together correctly. Comprehensive integration-test completion has not been established by the available test results; therefore, the above activities should be recorded as completed only after they have been executed and their results documented.

11.1.3 VALIDATION TESTING

Validation testing determines whether the software meets its intended functional requirements. It examines whether the system accepts valid inputs, rejects invalid inputs, and performs the operations expected by its users.
SnapTrace includes validation rules for reference selfies and event photograph uploads. Reference selfies must be in JPEG or PNG format and must not exceed 5 MB. The enrollment workflow requires exactly one detected face. Event photograph uploads accept JPEG or PNG files, with a maximum file size of 30 MB per file and a maximum of 10 files per request.
Validation testing should include the following cases:

- Submitting valid registration and login information.
- Uploading a reference selfie in a supported format.
- Attempting to upload a selfie exceeding the configured file-size limit.
- Submitting a selfie containing no detectable face or multiple faces.
- Uploading an unsupported image format.
- Attempting to upload event photographs exceeding the configured file-size or file-count limits.
- Submitting invalid credentials or unauthorized requests.

The implementation of these validation rules does not, by itself, demonstrate that every case has been tested successfully. Actual results should be recorded after executing the relevant test cases.

11.1.4 SYSTEM TESTING

System testing evaluates the application as a complete system. It examines whether the frontend, backend, database, authentication mechanisms, image-processing operations, and facial-recognition workflow function together as intended.
For SnapTrace, system testing should cover the complete workflow:

- 1. A user registers and logs in to the application.
- 2. The user submits a reference selfie for facial enrollment.

- 3. A photographer uploads event photographs.
- 4. The backend accepts the uploaded photographs and initiates background processing.
- 5. The facial-recognition pipeline detects faces and compares their descriptors with registered user descriptors.
- 6. Potential matches are associated with the relevant users.
- 7. Users access their personalized galleries.
- 8. Authorized administrators access the available management and reporting functions.

The system uses an Euclidean-distance threshold of 0.50 or lower to identify potential facial matches. This is a configured matching criterion and is not a measured accuracy percentage.
System testing should also examine error handling, authorization restrictions, database consistency, and the handling of uploaded media. Although the backend unit tests have passed, comprehensive end-to-end system testing has not been established by the available evidence.

11.1.5 OUTPUT TESTING

Output testing verifies whether the information generated by an application is correct, accessible, and presented in the expected format. It examines the results produced by different application operations. SnapTrace generates several types of output, including authentication responses, photograph upload responses, personalized galleries, processed images, thumbnails, administrative information, and JSON reports.
Output testing should verify the following:

- Successful authentication produces the expected response.
- Invalid authentication attempts are rejected appropriately.
- Accepted photograph uploads return the expected HTTP response.
- Processed photographs and thumbnails can be retrieved and displayed correctly.
- Personalized galleries display photographs associated with the appropriate user.
- JSON reports are generated in the expected format.
- Report metadata is stored correctly in the database.
- Errors are communicated through appropriate responses.

In SnapTrace, an HTTP 201 response indicates that the upload request has been accepted and processed according to the upload operation. It does not necessarily mean that background facial matching has finished. Output testing must therefore distinguish between successful upload acceptance and completion of the matching workflow.
Comprehensive output-test results have not been established by the available evidence and should be documented after the relevant checks are performed.

11.1.6 USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) evaluates whether the system meets the practical requirements and expectations of its intended users. It typically involves representative users performing common tasks and reviewing whether the application supports their needs.

For SnapTrace, the intended user groups include registered users, photographers, and administrators. Acceptance testing can be conducted using representative scenarios such as:

- Registering an account and logging in.
- Submitting a reference selfie.
- Uploading event photographs.
- Checking whether matching photographs appear in a personalized gallery.
- Reviewing available administrative functions.
- Generating and accessing reports.
- Evaluating the clarity of error messages and the ease of navigating the application.

During UAT, the tester should record the test scenario, expected result, actual result, status, and any issues encountered. Feedback should be used to identify usability problems and areas requiring improvement.

Formal user acceptance testing with documented results has not been established in the available project information. Therefore, these scenarios are presented as a proposed acceptance-testing procedure rather than completed UAT.

11.1.7 TESTING SUMMARY

The SnapTrace project includes automated backend testing using Vitest. The recorded execution contains five test suites and 44 tests, all of which passed. This demonstrates successful execution of the behaviours covered by those tests.

The current test evidence does not establish that all modules and workflows have been comprehensively tested. Further verification is required for integration between application components, the complete facial-recognition workflow, asynchronous image processing, frontend interactions, and end-to-end gallery distribution.

Performance testing, formal security assessment, user acceptance testing, and biometric accuracy benchmarking also require separate evaluation before the application can be considered fully verified for production deployment.

Overall, the completed backend unit tests provide a foundation for software quality assurance. Additional testing and documented results will strengthen confidence in SnapTrace's functional correctness, reliability, security, and suitability for real-world event media management.

NORMALIZATION

12. NORMALIZATION

Database normalization is the process of organizing data into tables to reduce unnecessary duplication and improve data consistency. It involves dividing data into related tables and establishing relationships between them using primary keys and foreign keys. Normalization helps minimize data redundancy, prevent insertion, update, and deletion anomalies, and maintain the integrity of stored information.

In SnapTrace, MySQL is used to manage user accounts, user settings, uploaded image information, personalized gallery associations, administrative audit records, reports, and password-reset tokens. The database contains seven principal tables: users, user_settings, images, user_gallery_photos, admin_audit_log, reports, and password_reset_tokens.

The database structure uses primary keys to identify records and relationships to connect related information. The following normal forms provide a basis for understanding the organization of the SnapTrace database.

The normal forms considered are:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)

12.1 FIRST NORMAL FORM (1NF)

A relation is in First Normal Form when each field contains a single value from its defined domain, each row can be uniquely identified, and repeating groups or multivalued attributes are not stored as repeated columns.

In SnapTrace, the database uses individual records to store user accounts, images, user settings, gallery associations, reports, audit activities, and password-reset tokens. Primary keys identify individual records in the tables.

For example, the users table stores account information, while the images table stores information about uploaded photographs. The user_gallery_photos table stores individual associations between users and matched images instead of storing a list of image associations in a single relational field.

The face_embedding column in the users table stores a facial descriptor as JSON data. Although JSON is used to represent the descriptor, this does not automatically establish that the entire table satisfies every 1NF requirement. Formal verification depends on the intended relational model and how the field is treated.

12.2 SECOND NORMAL FORM (2NF)

A relation is in Second Normal Form when it satisfies First Normal Form and every non-key attribute depends on the whole candidate key rather than only part of a composite candidate key. This condition is especially important for tables that use composite keys.

In SnapTrace, the user_gallery_photos table associates users with images and includes attributes such as match_confidence and matched_at. The combination of user_id and photo_id is subject to a unique constraint, preventing duplicate associations for the same user and image.

The attributes describing a particular gallery association belong to that association rather than independently to the user or image. The users and images tables separately store user-related and image-related information.

This structure helps avoid unnecessary duplication of user and image details within the gallery association table. A complete determination of 2NF status would require checking the candidate keys and functional dependencies of every table.

12.3 THIRD NORMAL FORM (3NF)

A relation is in Third Normal Form when it satisfies Second Normal Form and its non-key attributes do not have inappropriate transitive dependencies on a candidate key. In practical terms, non-key attributes should describe the entity represented by the table rather than depend on other non-key attributes.

SnapTrace separates information according to its purpose. The users table stores account-related information, user_settings stores user preferences, and images stores uploaded-image information. The user_gallery_photos table records the relationship between a user and an image. Similarly, password_reset_tokens stores password-reset information, admin_audit_log records administrative activities, and reports maintains report-related metadata.

This separation supports organized data storage and reduces the need to repeat the same information across multiple tables. Relationships between tables allow the application to retrieve associated data when required.

However, the actual normalization status of the entire database cannot be confirmed solely from the table names and descriptions. It requires examining the complete column definitions, candidate keys, and functional dependencies. Therefore, the SnapTrace database is designed using relational separation and relationships that support normalization, but universal compliance with 3NF should only be claimed after a formal schema review.

FUTURE ENHANCEMENT

13. FUTURE ENHANCEMENT

SnapTrace offers several opportunities for future enhancement in functionality, scalability, security, and user experience. A dedicated mobile application or Progressive Web Application (PWA) would let users and photographers access the system conveniently from smartphones and tablets, simplifying selfie submission, photograph browsing, and gallery access. Cloud-based object storage would help manage large photograph collections as the number of users and images grows, and a durable background-processing system using tools such as Redis and BullMQ would make photograph-processing jobs more reliable during large events or server restarts.

The facial-recognition module could be improved through systematic accuracy evaluation using representative datasets and documented performance metrics. Further work could include optimizing image-processing workflows, handling low-quality photographs better, and evaluating alternative face-matching methods. Any change to the matching algorithm should be validated before deployment to ensure it improves performance without introducing unacceptable matching errors. For larger deployments, vector indexing technologies such as FAISS or Milvus could be evaluated to speed up facial-descriptor searches across large user collections.

Security and privacy could be strengthened by storing password-reset tokens as cryptographic hashes, restricting direct access to uploaded media, and improving the management of signing secrets. Additional measures include multi-factor authentication, configurable data-retention policies, automated deletion of expired data, and clearer consent and privacy-management workflows, supporting more responsible handling of reference selfies, facial descriptors, and event photographs.

The user experience and administration could be enhanced with real-time processing-status indicators and notifications, along with a more advanced administrative dashboard showing statistics on uploads, processing failures, gallery associations, and system usage. Multi-language support could also make the application more accessible to users from different linguistic backgrounds.

Finally, comprehensive frontend and end-to-end automated testing, load testing, security assessment, and usability evaluation with representative users would provide stronger evidence of reliability and production readiness. By introducing these enhancements incrementally, SnapTrace could evolve into a more scalable, secure, and efficient platform for managing and distributing event photographs.

CONCLUSION

14. CONCLUSION

The **SnapTrace** project provides an intelligent and user-friendly solution for managing event photographs using facial recognition technology. The system addresses the common difficulty of finding individual photographs among large collections of event images by enabling users to access photographs that match their registered facial images. It simplifies the process of organizing, searching, and retrieving event photographs through a centralized web-based platform.

The system integrates modern web technologies, including **React, Vite, Node.js, Express.js, and MySQL**, to support user authentication, reference selfie registration, event photo uploads, facial recognition-based matching, personalized photo galleries, and administrative management. It also provides features for photographers to upload event photographs and for administrators to manage users, monitor system activities, and generate reports. These functionalities demonstrate the practical application of web development, database management, and facial recognition concepts in solving a real-world problem.

The development of SnapTrace demonstrates how automation and biometric image processing can improve the accessibility and organization of digital photographs. The system establishes a foundation for simplifying photo discovery and reducing the need for manual searching. However, the effectiveness of facial matching depends on factors such as image quality, lighting conditions, facial visibility, and the reliability of the recognition process. Appropriate attention to authentication, data protection, and the handling of facial information is also essential for responsible system operation.

In conclusion, **SnapTrace represents a practical application of intelligent image management and facial recognition in event photography**. The project provides a foundation for future enhancements, including improved recognition performance, more efficient processing of large photo collections, enhanced privacy controls, and expanded reporting capabilities. With further testing and development, SnapTrace can evolve into a more comprehensive platform for managing and retrieving event photographs efficiently.

REFERENCES

15. REFERENCES

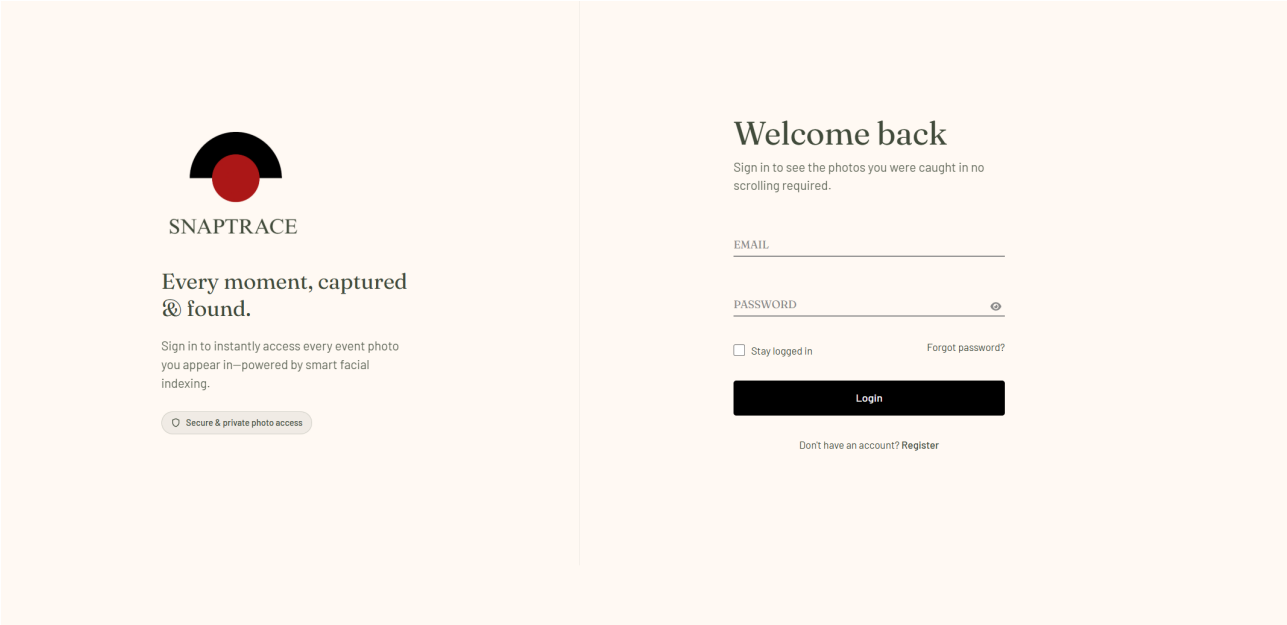
1. React Documentation. *React – The Library for Web and Native User Interfaces.* Available at: `https://react.dev/`
2. Vite Documentation. *Vite – Next Generation Frontend Tooling.* Available at: `https://vite.dev/`
3. Node.js Documentation. *Node.js – JavaScript Runtime Environment.* Available at: `https://nodejs.org/docs/`
4. Express.js Documentation. *Express – Node.js Web Application Framework.* Available at: `https://expressjs.com/`
5. MySQL Documentation. *MySQL Reference Manual.* Available at: `https://dev.mysql.com/doc/`
6. OpenCV Documentation. *OpenCV – Open Source Computer Vision Library.* Available at: `https://docs.opencv.org/`
7. MDN Web Docs. *HTML, CSS, and JavaScript Web Development Documentation.* Available at: `https://developer.mozilla.org/`
8. OWASP Foundation. *OWASP Top 10: Web Application Security Risks.* Available at: `https://owasp.org/www-project-top-ten/`
9. Schroff, F., Kalenichenko, D., and Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering.* Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 815–823. `https://doi.org/10.1109/CVPR.2015.7298682`
10. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications.* 2nd edition. Springer. Available at: `https://szeliski.org/Book/`

APPENDIX

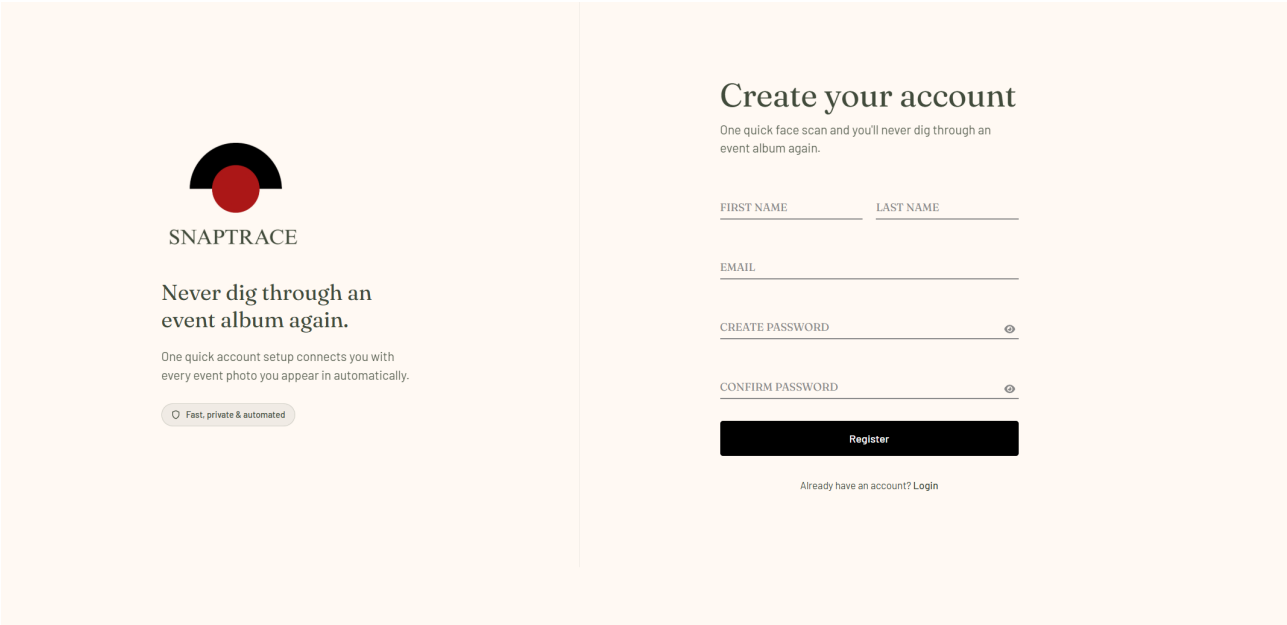
16. APPENDIX

16.1 SCREENSHOTS

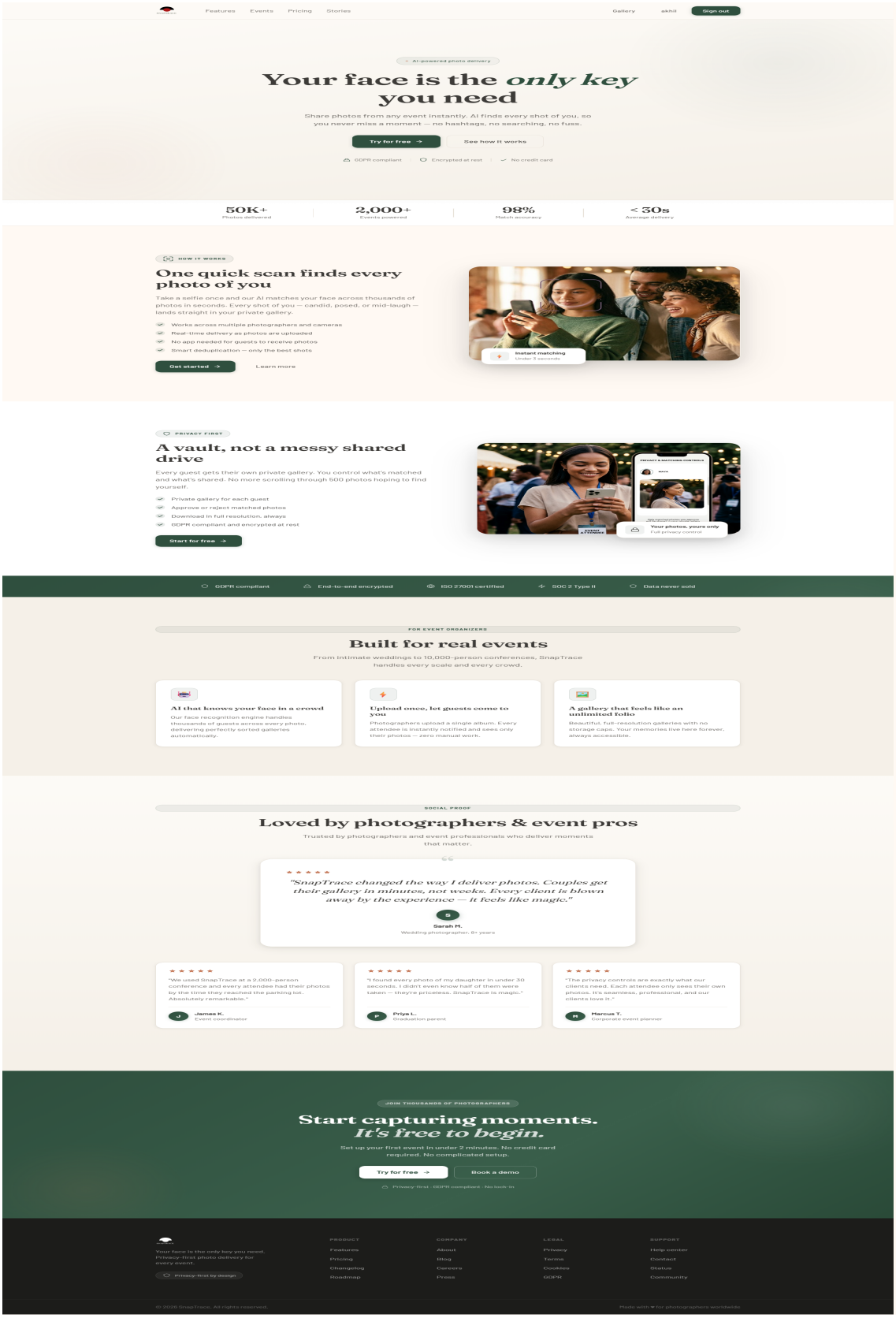
Login page



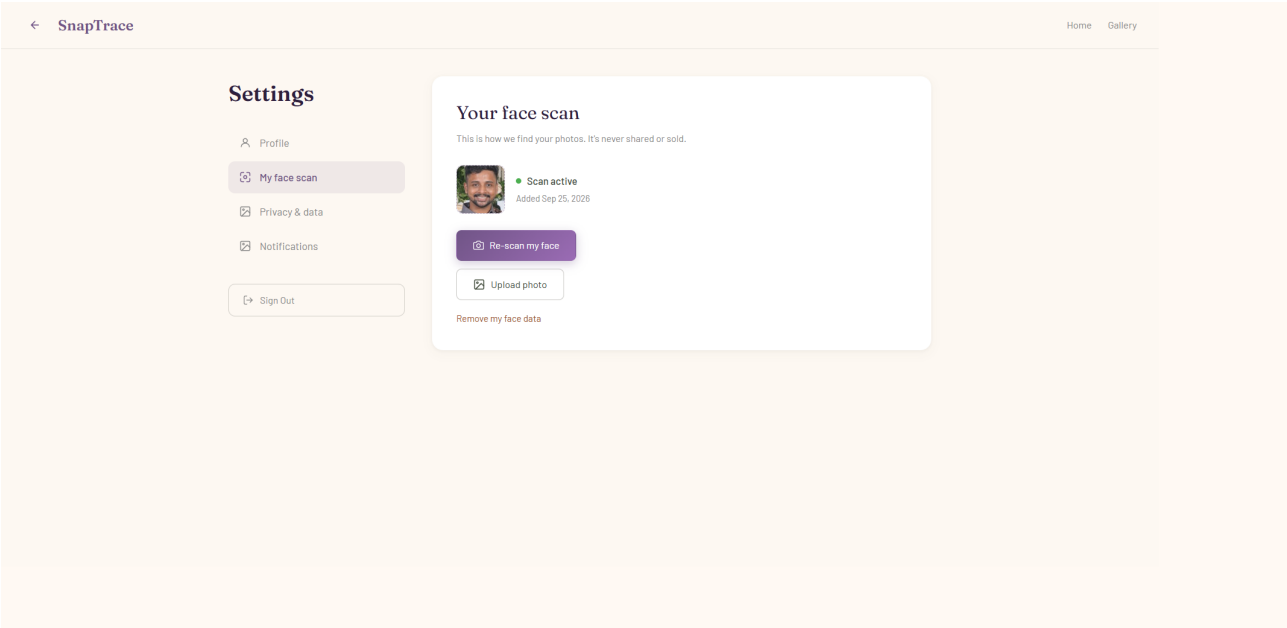
Register Page



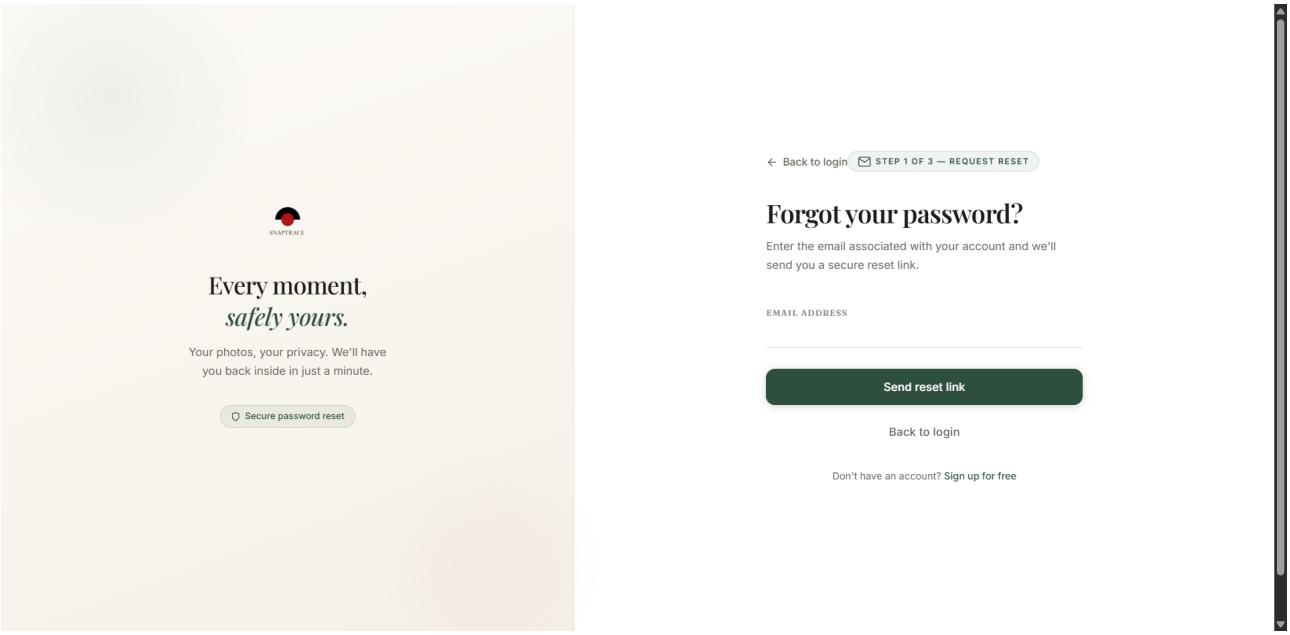
Home Page


Features
Events
Pricing
Stories
Gallery
akhil
Sign out
AI-powered photo delivery
Your face is the only key you need
Share photos from any event instantly. AI finds every shot of you, so you never miss a moment — no hashtags, no searching, no fuss.
Try for free →
See how it works
GDPR compliant
Encrypted at rest
No credit card
50K+
Photos delivered
2,000+
Events powered
98%
Match accuracy
< 30s
Average delivery
HOW IT WORKS
One quick scan finds every photo of you
Take a selfie once and our AI matches your face across thousands of photos in seconds. Every shot of you — candid, posed, or mid-laugh — lands straight in your private gallery.
Works across multiple photographers and cameras
Real-time delivery as photos are uploaded
No app needed for guests to receive photos
Smart deduplication — only the best shots
Get started →
Learn more
Instant matching
Under 3 seconds
PRIVACY FIRST
A vault, not a messy shared drive
Every guest gets their own private gallery. You control what's matched and what's shared. No more scrolling through 500 photos hoping to find yourself.
Private gallery for each guest
Approve or reject matched photos
Download in full resolution, always
GDPR compliant and encrypted at rest
Start for free →
Your photos, yours only
Full privacy control
GDPR compliant
End-to-end encrypted
ISO 27001 certified
SOC 2 Type II
Data never sold
FOR EVENT ORGANIZERS
Built for real events
From intimate weddings to 10,000-person conferences, SnapTrace handles every scale and every crowd.
AI that knows your face in a crowd
Our face recognition engine handles thousands of guests across every photo, delivering perfectly sorted galleries automatically.
Upload once, let guests come to you
Photographers upload a single album. Every attendee is instantly notified and sees only their photos — zero manual work.
A gallery that feels like an unlimited folio
Beautiful, full-resolution galleries with no storage caps. Your memories live here forever, always accessible.
SOCIAL PROOF
Loved by photographers & event pros
Trusted by photographers and event professionals who deliver moments that matter.
"SnapTrace changed the way I deliver photos. Couples get their gallery in minutes, not weeks. Every client is blown away by the experience — it feels like magic."
Sarah M.
Wedding photographer, 6+ years
"We used SnapTrace at a 2,000-person conference and every attendee had their photos by the time they reached the parking lot. Absolutely remarkable."
James K.
Event coordinator
"I found every photo of my daughter in under 30 seconds. I didn't even know half of them were taken — they're priceless. SnapTrace is magic."
Priya L.
Graduation parent
"The privacy controls are exactly what our clients need. Each attendee only sees their own photos. It's seamless, professional, and our clients love it."
Marcus T.
Corporate event planner
JOIN THOUSANDS OF PHOTOGRAPHERS
Start capturing moments. It's free to begin.
Set up your first event in under 2 minutes. No credit card required. No complicated setup.
Try for free →
Book a demo
Privacy-first · GDPR compliant · No lock-in
Your face is the only key you need. Privacy-first photo delivery for every event.
Privacy-first by design
PRODUCT
Features
Pricing
Changelog
Roadmap
COMPANY
About
Blog
Careers
Press
LEGAL
Privacy
Terms
Cookies
GDPR
SUPPORT
Help center
Contact
Status
Community
© 2026 SnapTrace. All rights reserved.
Made with ♥ for photographers worldwide

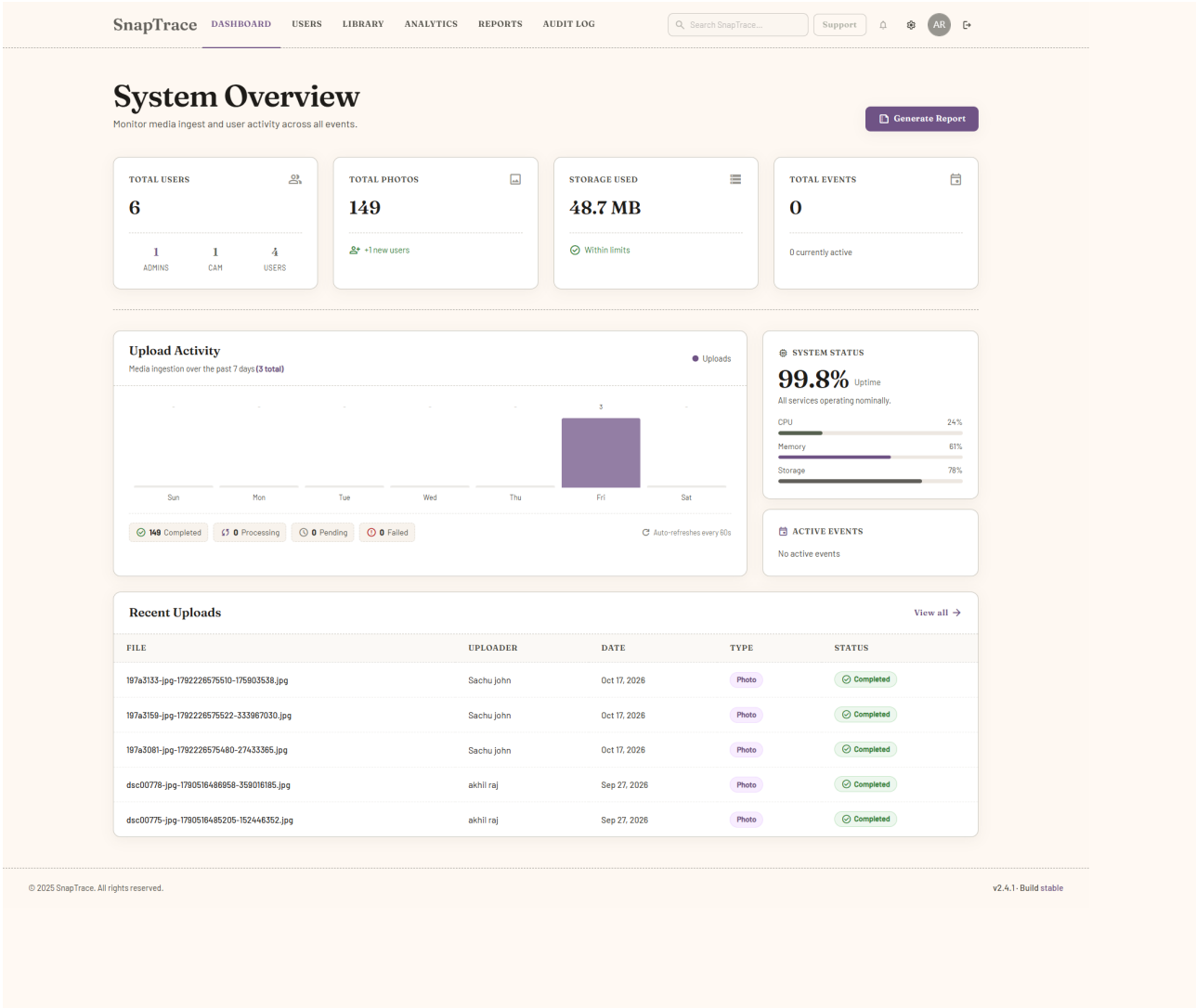
Face scan



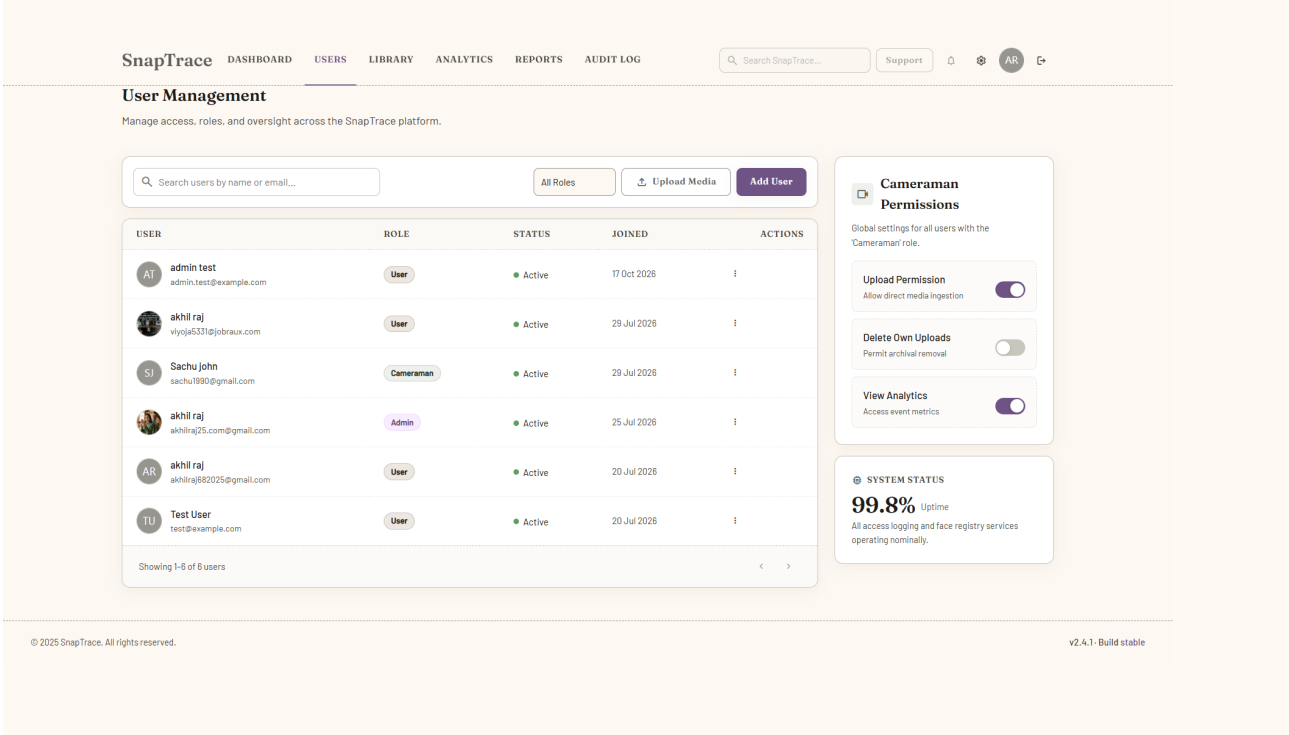
Forgot your password



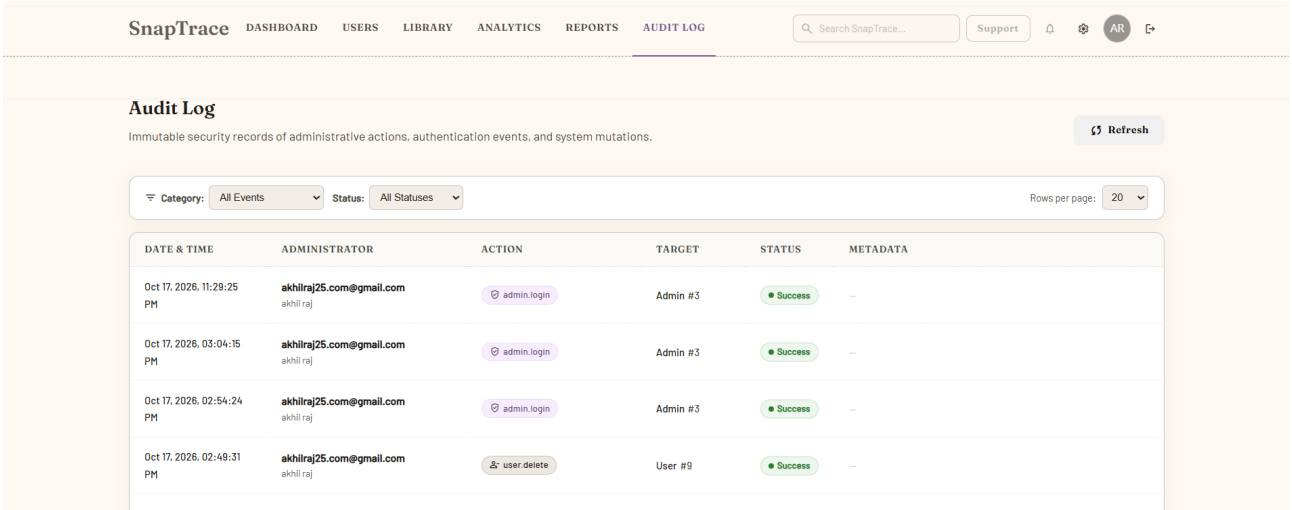
Admin - dashboard



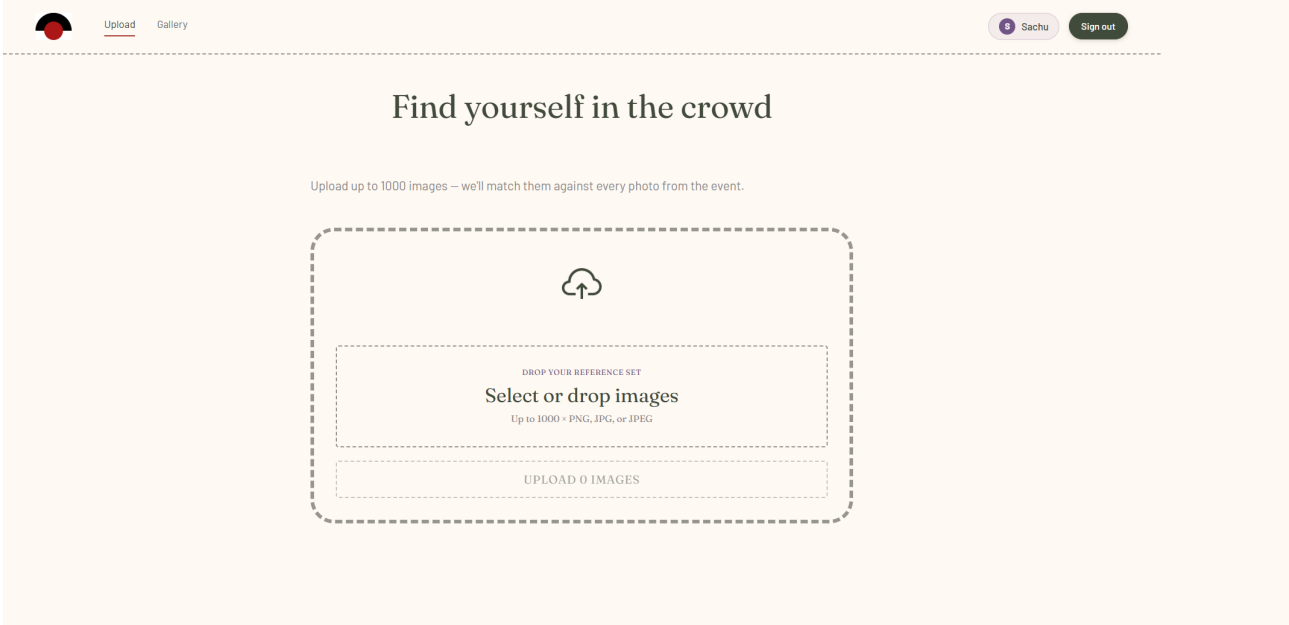
Admin - user management



Admin - Audit log



Cameraman - uploading page



16.2 SOURCE CODE

Frontend Component: src/LoginForm/LoginForm.jsx

```
import { useState } from 'react';
import { Navigate, useLocation } from 'react−router−dom';
import { loginUser } from '../api/auth';
import { useAuth } from '../context/AuthContext';
import DirectionalTransition from '../components/DirectionalTransition';
import { useTransitionNavigate } from '../hooks/useTransitionNavigate';
import TransitionLink from '../components/TransitionLink';
import logoImg from '../assets/logo.png';
import './Loginform.css';
import Input from '../components/Input';

const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [stayLoggedIn, setStayLoggedIn] = useState(false);
  const location = useLocation();
  const navigate = useTransitionNavigate();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const { login, user, loading: authLoading } = useAuth();

  if (!authLoading && user) {
    if (user.role === 'admin') return <Navigate to=”/admin” replace />;
    if (user.role === 'photographer') return <Navigate to=”/upload” replace />;
    return <Navigate to=”/gallery” replace />;
  }

  const handleLogin = async (e) => {
    if (e) e.preventDefault();
    if (loading) return;
    setError('');

    const trimmedEmail = email.trim();

    if (!trimmedEmail || !password) {
```

```
        setError('Please enter your email and password.');
        return;
    }

    if (!EMAIL_REGEX.test(trimmedEmail)) {
        setError('Please enter a valid email address.');
        return;
    }

    setLoading(true);
    try {
        const data = await loginUser({ email: trimmedEmail, password });
        login(data.token, data.user, stayLoggedIn);

        const from = location.state?.from?.pathname;

        if (data.user.role === 'admin') {
            navigate(from || '/admin');
        } else if (data.user.role === 'photographer' || data.user.role === 'cameraman') {
            if (from && (from === '/upload' || from.startsWith('/gallery') || from === '/settings')) {
                navigate(from);
            } else {
                navigate('/upload');
            }
        } else {
            if (from && (from.startsWith('/gallery') || from === '/settings')) {
                navigate(from);
            } else {
                navigate('/gallery');
            }
        }
    } catch (err) {
        setError(err.message || 'Incorrect email or password.');
    } finally {
        setLoading(false);
    }
};

return (
    <DirectionalTransition>
    <div className="lf__page">
        <div className="lf__container">
            <div className="lf__left">
```

```
          <div className="lf__left-content">
            <TransitionLink to="/home" className="lf__logo-link">
              <div className="lf__logo">
                <img src={logoImg} alt="SnapTrace" />
              </div>
            </TransitionLink>
            <h2 className="lf__brand-tagline">
              Every moment, captured & found.
            </h2>
            <p className="lf__brand-desc">
              Sign in to instantly access every event photo you appear in---powered by smart facial
indexing.
            </p>
          </div>
        </div>

        <div className="lf__right">
          <div className="lf__form-container">
            <h1 className="lf__title">Welcome back</h1>
            <p className="lf__subtitle">Sign in to your SnapTrace account</p>

            {error && (
              <div className="lf__alert lf__alert--error" role="alert">
                <span>{error}</span>
              </div>
            )}

            <form onSubmit={handleLogin} noValidate className="lf__form">
              <Input
                id="login-email"
                label="Email address"
                type="email"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                placeholder="name@example.com"
                required
                autoFocus
              />

              <Input
                id="login-password"
                label="Password"
                type="password"
```

```
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    placeholder="Enter your password"
                    required
                  />

                  <div className="lf__row">
                    <label className="lf__checkbox-label">
                      <input
                        type="checkbox"
                        checked={stayLoggedIn}
                        onChange={(e) => setStayLoggedIn(e.target.checked)}
                      />
                      <span>Remember me</span>
                    </label>
                    <TransitionLink to="/forgot-password" className="lf__forgot-link">
                      Forgot password?
                    </TransitionLink>
                  </div>

                  <button type="submit" className="lf__btn-submit" disabled={loading}>
                    {loading ? 'Signing in...' : 'Sign In'}
                  </button>
                </form>

                <p className="lf__footer-text">
                  Don't have an account?{' '}
                  <TransitionLink to="/register" className="lf__link-accent">
                    Create an account
                  </TransitionLink>
                </p>
              </div>
            </div>
          </div>
        </div>
      </DirectionalTransition>
    );
}

export default LoginForm;
```

Backend Controller: backend/controllers/authController.js (login)

```
export async function login(req, res) {
  const { email, password } = req.body || {};

  const normalizedEmail = typeof email === 'string' ? email.trim().toLowerCase() : '';
  const pwd = typeof password === 'string' ? password : '';

  if (!normalizedEmail || !pwd) {
    return res.status(400).json({ message: 'Email and password are required.' });
  }

  let user;
  try {
    const [rows] = await pool.query(
      'SELECT id, first_name, last_name, email, password, role, status FROM users WHERE email = ?',
      [normalizedEmail]
    );
    if (rows.length === 0) {
      return res.status(401).json({ message: 'Invalid credentials.' });
    }
    user = rows[0];
  } catch (err) {
    console.error('[login lookup error]', err.message);
    return res.status(500).json({ message: 'Database error.' });
  }

  if (user.status === 'suspended') {
    return res.status(403).json({ message: 'Account is suspended.' });
  }
  if (user.status === 'inactive') {
    return res.status(403).json({ message: 'Account is inactive. Please contact support.' });
  }

  const isMatch = await bcrypt.compare(pwd, user.password);
  if (!isMatch) {
    if (user.role === 'admin') {
      await logAudit({
        adminId: user.id,
        action: 'admin.login_failed',
        targetType: 'admin',
        targetId: user.id,
        req,
        success: false,
        metadata: { reason: 'invalid_credentials' },
```

```
        });
      }
      return res.status(401).json({ message: 'Invalid credentials.' });
    }

    try {
      await pool.query(
        'UPDATE users SET last_login_at = NOW() WHERE id = ?',
        [user.id]
      );
    } catch (err) {
      console.error('[login update last_login_at error]', err.message);
    }

    if (user.role === 'admin') {
      await logAudit({
        adminId: user.id,
        action: 'admin.login',
        targetType: 'admin',
        targetId: user.id,
        req,
        success: true,
      });
    }

    const token = jwt.sign(
      { id: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );
    return res.json({
      message: 'Login successful.',
      token,
      user: {
        id: user.id,
        first_name: user.first_name,
        last_name: user.last_name,
        email: user.email,
        role: user.role,
      },});}
```

Frontend Component: src/RegisterForm/RegisterForm.jsx

```
import { useState } from 'react';
import { Navigate } from 'react-router-dom';
import { registerUser } from '../api/auth';
import { useAuth } from '../context/AuthContext';
import DirectionalTransition from '../components/DirectionalTransition';
import { useTransitionNavigate } from '../hooks/useTransitionNavigate';
import logoImg from '../assets/logo.png';
import TransitionLink from '../components/TransitionLink';
import Input from '../components/Input';
import './Resgform.css';

const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function calcStrength(pw) {
  if (!pw) return 0;
  let score = 0;
  if (pw.length >= 8) score++;
  if (pw.length >= 12) score++;
  if (/[A-Z]/.test(pw)) score++;
  if (/[0-9]/.test(pw)) score++;
  if (/[^A-Za-z0-9]/.test(pw)) score++;
  return Math.min(score, 4);
}

const STRENGTH_META = [
  null,
  { label: 'Weak',   cls: 'weak' },
  { label: 'Fair',   cls: 'medium' },
  { label: 'Good',   cls: 'medium' },
  { label: 'Strong', cls: 'strong' },
];

function RegisterForm() {
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword]= useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const navigate = useTransitionNavigate();
  const { login, user, loading: authLoading } = useAuth();
```

```
if (!authLoading && user) {
  if (user.role === 'admin') return <Navigate to="/admin" replace />;
  if (user.role === 'photographer') return <Navigate to="/upload" replace />;
  return <Navigate to="/gallery" replace />;
}

const strength = calcStrength(password);
const strengthMeta = STRENGTH_META[strength];

const requirements = [
  { label: 'At least 8 characters', met: password.length >= 8 },
  { label: 'One uppercase letter (A–Z)', met: /[A–Z]/.test(password) },
  { label: 'One number (0–9)', met: /[0–9]/.test(password) },
  { label: 'One special character', met: /[^A–Za–z0–9]/.test(password) },
  { label: 'Passwords match', met: Boolean(confirmPassword) && confirmPassword === password },
];

const handleRegister = async (e) => {
  if (e) e.preventDefault();
  if (loading) return;
  setError('');

  const trimmedFirst = firstName.trim();
  const trimmedLast = lastName.trim();
  const trimmedEmail = email.trim().toLowerCase();

  if (!trimmedFirst || !trimmedLast || !trimmedEmail || !password || !confirmPassword) {
    setError('All fields are required.');
    return;
  }

  if (!EMAIL_REGEX.test(trimmedEmail)) {
    setError('Please enter a valid email address.');
    return;
  }

  if (password.length < 8) {
    setError('Password must be at least 8 characters long.');
    return;
  }

  if (password !== confirmPassword) {
    setError('Passwords do not match.');
```

```
      return;
    }

    setLoading(true);
    try {
      const data = await registerUser({
        first_name: trimmedFirst,
        last_name: trimmedLast,
        email: trimmedEmail,
        password,
      });
      login(data.token, data.user);
      navigate('/gallery');
    } catch (err) {
      setError(err.message || 'Registration failed. Please try again.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <DirectionalTransition>
    <div className="rf__page">
      <div className="rf__container">
        <div className="rf__form-wrapper">
          <h1 className="rf__title">Create your account</h1>
          <p className="rf__subtitle">Join SnapTrace to find all your event photos</p>

          {error && <div className="rf__alert rf__alert--error">{error}</div>}

          <form onSubmit={handleRegister} className="rf__form" noValidate>
            <div className="rf__grid-2">
              <Input
                id="reg-fname"
                label="First name"
                type="text"
                value={firstName}
                onChange={(e) => setFirstName(e.target.value)}
                required
              />
              <Input
                id="reg-lname"
                label="Last name"
```

```
            type="text"
            value={lastName}
            onChange={(e) => setLastName(e.target.value)}
            required
          />
        </div>

        <Input
          id="reg-email"
          label="Email address"
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />

        <Input
          id="reg-password"
          label="Password"
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />

        {password && (
          <div className="rf__strength-box">
            <div className="rf__strength-meter">
              <div className={`rf__strength-bar rf__strength-bar--${strengthMeta?.cls || ''}`}
                style={{ width: `${(strength / 4) * 100}%` }} />
            </div>
            <span className="rf__strength-label">Strength: {strengthMeta?.label || 'Weak'}</span
>
          </div>
        )}

        <Input
          id="reg-confirm"
          label="Confirm password"
          type="password"
          value={confirmPassword}
          onChange={(e) => setConfirmPassword(e.target.value)}
          required
```

```
                />

                <ul className="rf__req-list">
                  {requirements.map((req, i) => (
                    <li key={i} className={req.met ? 'met' : 'unmet'}>
                      {req.met ? '\u2713' : '\u2022'} {req.label}
                    </li>
                  ))}
                </ul>

                <button type="submit" className="rf__submit-btn" disabled={loading}>
                  {loading ? 'Creating account...' : 'Create Account'}
                </button>
              </form>
            </div>
          </div>
        </div>
      </DirectionalTransition>
    );
}

export default RegisterForm;
```

Backend Controller: backend/controllers/authController.js (register)

```
export async function register(req, res) {
  const { first_name, last_name, email, password } = req.body || {};

  const firstName = typeof first_name === 'string' ? first_name.trim() : '';
  const lastName = typeof last_name === 'string' ? last_name.trim() : '';
  const normalizedEmail = typeof email === 'string' ? email.trim().toLowerCase() : '';
  const pwd = typeof password === 'string' ? password : '';

  if (!firstName || !lastName || !normalizedEmail || !pwd) {
    return res.status(400).json({ message: 'All fields are required.' });
  }

  if (!EMAIL_REGEX.test(normalizedEmail)) {
    return res.status(400).json({ message: 'Please provide a valid email address.' });
  }

  if (pwd.length < 8) {
    return res.status(400).json({ message: 'Password must be at least 8 characters long.' });
```

```
    }

    try {
      const [existing] = await pool.query(
        'SELECT id FROM users WHERE email = ?', [normalizedEmail]
      );
      if (existing.length > 0) {
        return res.status(409).json({ message: 'Email already in use.' });
      }
    } catch (err) {
      console.error('[register check error]', err.message);
      return res.status(500).json({ message: 'Database error.' });
    }

    let hashedPassword;
    try {
      hashedPassword = await bcrypt.hash(pwd, SALT_ROUNDS);
    } catch (err) {
      console.error('[register hash error]', err.message);
      return res.status(500).json({ message: 'Error processing credentials.' });
    }

    try {
      const [result] = await pool.query(
        'INSERT INTO users (first_name, last_name, email, password, role) VALUES (?, ?, ?, ?, ?)',
        [firstName, lastName, normalizedEmail, hashedPassword, 'user']
      );
      const userId = result.insertId;
      const token = jwt.sign(
        { id: userId, email: normalizedEmail, role: 'user' },
        process.env.JWT_SECRET,
        { expiresIn: '7d' }
      );
      return res.status(201).json({
        message: 'User registered successfully.',
        token,
        user: { id: userId, first_name: firstName, last_name: lastName, email: normalizedEmail, role: 'user' },
      });
    } catch (err) {
      console.error('[register insert error]', err.message);
      return res.status(500).json({ message: 'Could not create user.' });
    }
}
```

Frontend Component: src/UserSettings/FaceCapture.jsx

```
import { useState, useEffect, useRef, useCallback } from 'react';
import './FaceCapture.css';

function FaceScanBigIcon() {
 return (
  <svg viewBox="0 0 120 120" fill="none" xmlns="http://www.w3.org/2000/svg" className="fc-scan-
    icon">
   <path d="M10 30 L10 10 L30 10" stroke="#715587" strokeWidth="4" strokeLinecap="round"
    strokeLinejoin="round"/>
   <path d="M90 10 L110 10 L110 30" stroke="#715587" strokeWidth="4" strokeLinecap="round"
    strokeLinejoin="round"/>
   <path d="M110 90 L110 110 L90 110" stroke="#715587" strokeWidth="4" strokeLinecap="round"
    strokeLinejoin="round"/>
   <path d="M30 110 L10 110 L10 90" stroke="#715587" strokeWidth="4" strokeLinecap="round"
    strokeLinejoin="round"/>
   <ellipse cx="60" cy="55" rx="22" ry="26" stroke="#715587" strokeWidth="3"/>
   <circle cx="52" cy="50" r="3" fill="#715587"/>
   <circle cx="68" cy="50" r="3" fill="#715587"/>
   <path d="M51 62 Q60 70 69 62" stroke="#715587" strokeWidth="2.5" strokeLinecap="round" fill="
    none"/>
  </svg>
 );
}

export default function FaceCapture({ onCapture, onClose }) {
 const [phase, setPhase] = useState('idle');
 const [capturedBlob, setCapturedBlob] = useState(null);
 const [previewUrl, setPreviewUrl] = useState(null);
 const [errorMsg, setErrorMsg] = useState('');

 const videoRef = useRef(null);
 const canvasRef = useRef(null);
 const streamRef = useRef(null);
 const fileInputRef = useRef(null);

 const stopStream = useCallback(() => {
  if (streamRef.current) {
   streamRef.current.getTracks().forEach((track) => track.stop());
   streamRef.current = null;
  }
```

```
}, []);

useEffect(() => () => stopStream(), [stopStream]);

const startCamera = async () => {
  setErrorMsg('');
  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: { facingMode: 'user', width: { ideal: 720 }, height: { ideal: 720 } },
      audio: false,
    });
    streamRef.current = stream;
    setPhase('streaming');
  } catch (err) {
    setErrorMsg('Camera access was denied or is unavailable. Please upload a photo instead.');
    setPhase('error');
  }
};

useEffect(() => {
  if (phase === 'streaming' && videoRef.current && streamRef.current) {
    videoRef.current.srcObject = streamRef.current;
    videoRef.current.play().catch(() => {});
  }
}, [phase]);

const captureFrame = () => {
  const video = videoRef.current;
  const canvas = canvasRef.current;
  if (!video || !canvas) return;

  canvas.width = video.videoWidth || 640;
  canvas.height = video.videoHeight || 480;
  const ctx = canvas.getContext('2d');

  ctx.drawImage(video, 0, 0, canvas.width, canvas.height);

  canvas.toBlob((blob) => {
    if (!blob) return;
    const file = new File([blob], 'reference-face.jpg', { type: 'image/jpeg' });
    setCapturedBlob(file);
    setPreviewUrl(URL.createObjectURL(blob));
    stopStream();
```

```
    setPhase('preview');
  }, 'image/jpeg', 0.95);
};

const handleFileUpload = (e) => {
  const file = e.target.files?.[0];
  if (file) {
    setCapturedBlob(file);
    setPreviewUrl(URL.createObjectURL(file));
    setPhase('preview');
  }
};

const handleConfirm = () => {
  if (capturedBlob) {
    onCapture(capturedBlob);
    onClose();
  }
};

return (
  <div className="fc-modal-overlay">
    <div className="fc-modal-card">
      <h2 className="fc-title">Face Verification</h2>
      <p className="fc-subtitle">Register your face to automatically unlock your event photos</p>

      <div className="fc-viewport-frame">
        {phase === 'idle' && (
          <div className="fc-idle-pane">
            <FaceScanBigIcon />
            <button className="fc-btn fc-btn--primary" onClick={startCamera}>
              Open Camera
            </button>
          </div>
        )}

        {phase === 'streaming' && (
          <div className="fc-streaming-pane">
            <video ref={videoRef} className="fc-video-feed" playsInline muted />
            <div className="fc-guide-oval" />
            <div className="fc-scan-line-anim" />
          </div>
        )}
```

```
  {phase === 'preview' && previewUrl && (
    <div className="fc-preview-pane">
      <img src={previewUrl} alt="Face preview" className="fc-preview-img" />
    </div>
  )}

  {phase === 'error' && (
    <div className="fc-error-pane">
      <p className="fc-error-text">{errorMsg}</p>
    </div>
  )}
 </div>

 <canvas ref={canvasRef} style={{ display: 'none' }} />
 <input ref={fileInputRef} type="file" accept="image/jpeg,image/png" style={{ display: 'none' }}
onChange={handleFileUpload} />

 <div className="fc-action-bar">
  {phase === 'streaming' && (
    <button className="fc-btn fc-btn--capture" onClick={captureFrame}>
      Capture Selfie
    </button>
  )}

  {phase === 'preview' && (
    <div className="fc-btn-group">
      <button className="fc-btn fc-btn--ghost" onClick={() => { setPhase('idle'); setCapturedBlob(
null); }}>
        Retake
      </button>
      <button className="fc-btn fc-btn--primary" onClick={handleConfirm}>
        Confirm & Save
      </button>
    </div>
  )}

  {(phase === 'idle' || phase === 'error') && (
    <button className="fc-btn fc-btn--outline" onClick={() => fileInputRef.current?.click()}>
      Upload From Device
    </button>
  )}
```

```
        <button className="fc-btn fc-btn--cancel" onClick={onClose}>
          Cancel
        </button>
      </div>
    </div>
  </div>
 );
}
```

Backend Controller: backend/controllers/userController.js (uploadFaceScan)

```
export async function uploadFaceScan(req, res) {
  if (!req.file) {
    return res.status(400).json({ success: false, message: 'No file received.' });
  }
  try {
    const [rows] = await pool.execute(
      'SELECT face_scan_url FROM users WHERE id = ?',
      [req.user.id]
    );
    const oldUrl = rows[0]?.face_scan_url || null;

    const destFilename = `${req.user.id}-face-scan.jpg`;
    const destPath = path.join(faceScanDir, destFilename);
    await compressAndSave(req.file.path, destPath, 600);

    fs.unlink(req.file.path, () => { });

    let embedding;
    try {
      const result = await detectSingleFace(destPath);
      embedding = result.embedding;
    } catch (faceErr) {
      fs.unlink(destPath, () => { });
      return res.status(400).json({ success: false, message: faceErr.message });
    }

    const faceScanUrl = `/uploads/face-scans/${destFilename}`;
    await pool.execute(
      `UPDATE users
       SET face_scan_url = ?, face_scan_date = NOW(),
          face_embedding = ?, face_embedding_updated_at = NOW()
       WHERE id = ?`,
```

```
        [faceScanUrl, JSON.stringify(embedding), req.user.id]
      );

      if (oldUrl && oldUrl !== faceScanUrl) safeUnlink(oldUrl);

      enqueueUserRematch(req.user.id, embedding);

      const [updated] = await pool.execute(
        'SELECT face_scan_url, face_scan_date FROM users WHERE id = ?',
        [req.user.id]
      );
      return res.json({ success: true, data: updated[0] });
    } catch (err) {
      if (req.file?.path) fs.unlink(req.file.path, () => { });
      console.error('[uploadFaceScan error]', err.message);
      return res.status(500).json({ success: false, message: 'Failed to save face scan.' });
    }
}
```

Frontend Component: src/UploadForm/UploadForm.jsx

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useTransitionNavigate } from '../hooks/useTransitionNavigate';
import DirectionalTransition from '../components/DirectionalTransition';
import { apiUpload } from '../api/client';
import './Uploadform.css';

const ACCEPTED_TYPES = ['image/jpeg', 'image/jpg', 'image/png'];
const ACCEPTED_EXTS = ['jpg', 'jpeg', 'png'];
const MAX_FILES = 1000;

const isValidImage = (f) => {
  const ext = f.name.split('.').pop()?.toLowerCase();
  return ACCEPTED_TYPES.includes(f.type) || ACCEPTED_EXTS.includes(ext);
};

export default function MultipleFileUpload() {
  const { user, logout } = useAuth();
  const navigate = useTransitionNavigate();

  const [files, setFiles] = useState([]);
  const [dragActive, setDragActive] = useState(false);
```

```
const [uploading, setUploading] = useState(false);
const [globalMsg, setGlobalMsg] = useState({ text: '', type: '' });

const fileInputRef = useRef(null);

useEffect(() => () => files.forEach((f) => URL.revokeObjectURL(f.preview)), [files]);

const stageFiles = useCallback((incoming) => {
  setGlobalMsg({ text: '', type: '' });
  const rawList = Array.from(incoming);
  const valid = [];
  const rejected = [];

  for (const f of rawList) {
    if (!isValidImage(f)) { rejected.push(f.name); continue; }
    valid.push(f);
  }

  setFiles((prev) => {
    const slots = MAX_FILES − prev.length;
    const toAdd = valid.slice(0, slots);
    const entries = toAdd.map((f) => ({
      id: crypto.randomUUID(),
      file: f,
      preview: URL.createObjectURL(f),
      uploadStatus: 'pending',
    }));
    return [...prev, ...entries];
  });
}, []);

const handleUploadAll = async () => {
  const pendingFiles = files.filter(f => f.uploadStatus === 'pending');
  if (!pendingFiles.length) return;

  setUploading(true);
  const formData = new FormData();
  pendingFiles.forEach(item => {
    formData.append('images', item.file);
  });

  try {
    await apiUpload('/api/upload', formData);
```

```
    setGlobalMsg({ text: `Successfully uploaded ${pendingFiles.length} event photo(s)!`, type: 'success' });
    setFiles(prev => prev.map(f => ({ ...f, uploadStatus: 'done' })));
  } catch (err) {
    setGlobalMsg({ text: err.message || 'Upload failed.', type: 'error' });
  } finally {
    setUploading(false);
  }
};

return (
  <DirectionalTransition>
    <div className="uf-page">
      <div className="uf-container">
        <header className="uf-header">
          <h1 className="uf-title">Event Photograph Intake</h1>
          <p className="uf-subtitle">Upload and index multi-attendee event galleries</p>
        </header>

        {globalMsg.text && (
          <div className={`uf-alert uf-alert--${globalMsg.type}`}>{globalMsg.text}</div>
        )}

        <div
          className={`uf-dropzone ${dragActive ? 'uf-dropzone--active' : ''}`}
          onDragOver={(e) => { e.preventDefault(); setDragActive(true); }}
          onDragLeave={() => setDragActive(false)}
          onDrop={(e) => {
            e.preventDefault();
            setDragActive(false);
            stageFiles(e.dataTransfer.files);
          }}
          onClick={() => fileInputRef.current?.click()}
        >
          <p className="uf-drop-title">Drag & drop high-resolution photographs here</p>
          <p className="uf-drop-hint">Supports JPEG & PNG up to 30MB each</p>
          <input
            ref={fileInputRef}
            type="file"
            multiple
            accept="image/jpeg,image/png"
            style={{ display: 'none' }}
            onChange={(e) => stageFiles(e.target.files)}
          />
```

```
        </div>

        {files.length > 0 && (
          <div className="uf-preview-grid">
            {files.map((item) => (
              <div key={item.id} className="uf-preview-card">
                <img src={item.preview} alt="Thumbnail preview" className="uf-thumb" />
                <span className={`uf-status-pill uf-status-pill--${item.uploadStatus}`}>
                  {item.uploadStatus}
                </span>
              </div>
            ))}
          </div>
        )}

        {files.some(f => f.uploadStatus === 'pending') && (
          <div className="uf-action-row">
            <button className="uf-btn-upload" onClick={handleUploadAll} disabled={uploading}>
              {uploading ? 'Processing Batch...' : `Upload ${files.filter(f => f.uploadStatus === 'pending').length
      } Photos`}
            </button>
          </div>
        )}
      </div>
    </div>
  </DirectionalTransition>
 );
}
```

Backend Controller: backend/controllers/uploadController.js (uploadImages)

```
export async function uploadImages(req, res) {
  if (!req.files || req.files.length === 0) {
    return res.status(400).json({ message: 'No files received.' });
  }
  const inserted = [];
  const errors = [];

  for (const f of req.files) {
    const ext = path.extname(f.filename).toLowerCase();
    const origPath = path.join(uploadDir, f.filename);
    const filepath = `/uploads/${f.filename}`;
    let thumbFilename = '';
```

```
let thumbnail_path = '';
let photoId = null;

try {
  const fileStats = fs.statSync(origPath);
  const [result] = await pool.query(
    `INSERT INTO images (filename, filepath, thumbnail_path, original_name, mime_type, file_size,
uploaded_by, status)
     VALUES (?, ?, '', ?, ?, ?, ?, 'processing')`,
    [f.filename, filepath, f.originalname || null, f.mimetype || null, fileStats.size || 0, req.user?.id || null]
  );
  photoId = result.insertId;
} catch (dbErr) {
  console.error('[db insert error]', dbErr.message);
  try { fs.unlinkSync(origPath); } catch (_) {}
  errors.push(f.filename);
  continue;
}

let processingOk = true;
try {
  await compressOriginal(origPath, ext);
  thumbFilename = await generateThumbnail(origPath, f.filename);
  thumbnail_path = `/uploads/thumbnails/${thumbFilename}`;
} catch (sharpErr) {
  console.error('[sharp error]', sharpErr.message);
  processingOk = false;
}

if (processingOk) {
  try {
    await pool.query(
      `UPDATE images SET status = 'completed', thumbnail_path = ? WHERE id = ?`,
      [thumbnail_path, photoId]
    );
    inserted.push({ id: photoId, filename: f.filename, path: filepath, thumbnail_path });

    enqueuePhotoMatch(photoId, origPath);
  } catch (updateErr) {
    console.error('[db update error]', updateErr.message);
    errors.push(f.filename);
  }
} else {
```

```
        try {
          await pool.query('UPDATE images SET status = 'failed' WHERE id = ?', [photoId]);
          fs.unlinkSync(origPath);
        } catch (_) {}
        errors.push(f.filename);
      }
    }

    if (inserted.length === 0) {
      return res.status(500).json({ message: 'Upload succeeded but could not save records.' });
    }

    return res.status(201).json({
      message: `${inserted.length} file(s) uploaded successfully.`,
      files: inserted,
    });
}
```

Core Service: backend/services/faceRecognition.js

```
import path from 'node:path';
import fs from 'node:fs';
import { fileURLToPath } from 'node:url';
import sharp from 'sharp';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const MODEL_PATH = path.join(__dirname, '..', 'models');

export const MATCH_THRESHOLD = parseFloat(process.env.FACE_MATCH_THRESHOLD ?? '0.50');

let _modelsLoaded = false;
let _faceapi = null;
let _tf = null;

async function _ensureLoaded() {
  if (_faceapi) return;

  const tfCore = await import('@tensorflow/tfjs-core');
  await import('@tensorflow/tfjs-backend-wasm');

  await tfCore.setBackend('wasm');
  await tfCore.ready();
```

```
  _tf = tfCore;
  _faceapi = await import('@vladmandic/face-api/dist/face-api.node-wasm.js');
}

export async function loadFaceModels() {
  if (_modelsLoaded) return;

  if (!fs.existsSync(MODEL_PATH)) {
    throw new Error(`Models directory not found: ${MODEL_PATH}.`);
  }

  await _ensureLoaded();

  await Promise.all([
    _faceapi.nets.ssdMobilenetv1.loadFromDisk(MODEL_PATH),
    _faceapi.nets.faceLandmark68Net.loadFromDisk(MODEL_PATH),
    _faceapi.nets.faceRecognitionNet.loadFromDisk(MODEL_PATH),
  ]);

  _modelsLoaded = true;
  console.log(' Face-API models loaded successfully (WASM runtime)');
}

async function imageToInput(imagePath) {
  const { data, info } = await sharp(imagePath)
    .resize(640, 640, { fit: 'inside', withoutEnlargement: true })
    .removeAlpha()
    .raw()
    .toBuffer({ resolveWithObject: true });

  const float32 = new Float32Array(data.length);
  for (let i = 0; i < data.length; i++) float32[i] = data[i];

  return _tf.tensor3d(float32, [info.height, info.width, 3]);
}

export async function detectSingleFace(imagePath) {
  if (!_modelsLoaded) await loadFaceModels();

  const input = await imageToInput(imagePath);
  const options = new _faceapi.SsdMobilenetv1Options({ minConfidence: 0.5 });

  try {
```

```
    const detections = await _faceapi
      .detectAllFaces(input, options)
      .withFaceLandmarks()
      .withFaceDescriptors();

    if (detections.length === 0) {
      throw new Error('No face detected. Please upload a clear, front-facing photo.');
    }
    if (detections.length > 1) {
      throw new Error(`${detections.length} faces detected. Reference photo must contain only your face.`)
    ;
    }

    const det = detections[0];
    return {
      embedding: Array.from(det.descriptor),
      box: {
        x: det.detection.box.x,
        y: det.detection.box.y,
        width: det.detection.box.width,
        height: det.detection.box.height,
      },
    };
  } finally {
    input.dispose();
  }
}

export async function detectAllFaces(imagePath) {
  if (!_modelsLoaded) await loadFaceModels();

  const input = await imageToInput(imagePath);
  const options = new _faceapi.SsdMobilenetv1Options({ minConfidence: 0.5 });

  try {
    const detections = await _faceapi
      .detectAllFaces(input, options)
      .withFaceLandmarks()
      .withFaceDescriptors();

    return detections.map((d) => ({
      embedding: Array.from(d.descriptor),
      box: {
```

```
            x: d.detection.box.x,
            y: d.detection.box.y,
            width: d.detection.box.width,
            height: d.detection.box.height,
          },
        }));
    } finally {
      input.dispose();
    }
}
export function euclideanDistance(a, b) {
    if (!a || !b || a.length !== b.length) return Infinity;
    let sum = 0;
    for (let i = 0; i < a.length; i++) {
      const d = a[i] - b[i];
      sum += d * d;}
    return Math.sqrt(sum);}
```

Asynchronous Face-Matching Queue: backend/services/matchingQueue.js

```
import fs from 'node:fs';
import path from 'node:path';
import pool from '../config/db.js';
import { detectAllFaces, euclideanDistance, MATCH_THRESHOLD } from './faceRecognition.js';

const _queue = [];
let _active = false;

export function enqueuePhotoMatch(photoId, absoluteImagePath) {
    _queue.push({ type: 'photo', photoId, imagePath: absoluteImagePath });
    if (!_active) _processNext();
}

export function enqueueUserRematch(userId, embedding) {
    _queue.push({ type: 'user', userId, embedding });
    if (!_active) _processNext();
}

async function _processNext() {
    if (_queue.length === 0) { _active = false; return; }
    _active = true;

    const job = _queue.shift();
```

```
    try {
      if (job.type === 'user') {
        await _rematchUser(job.userId, job.embedding);
      } else {
        await _matchPhoto(job.photoId, job.imagePath);
      }
    } catch (err) {
      console.error(`[matchingQueue] Error processing job:`, err.message);
    }

    setImmediate(_processNext);
}

async function _matchPhoto(photoId, imagePath) {
    if (!fs.existsSync(imagePath)) return;

    const faces = await detectAllFaces(imagePath);
    if (!faces.length) return;

    const [users] = await pool.execute(
      'SELECT id, face_embedding FROM users WHERE face_embedding IS NOT NULL'
    );
    if (!users.length) return;

    const seen = new Set();

    for (const face of faces) {
      for (const user of users) {
        if (seen.has(user.id)) continue;

        let storedEmbedding;
        try {
          storedEmbedding = typeof user.face_embedding === 'string'
            ? JSON.parse(user.face_embedding)
            : user.face_embedding;
        } catch { continue; }

        const dist = euclideanDistance(face.embedding, storedEmbedding);

        if (dist <= MATCH_THRESHOLD) {
          seen.add(user.id);
          const confidence = Math.max(0, 1 - dist);
```

```
            await pool.execute(
              `INSERT INTO user_gallery_photos (user_id, photo_id, match_confidence)
               VALUES (?, ?, ?)
               ON DUPLICATE KEY UPDATE
                 match_confidence = GREATEST(match_confidence, VALUES(match_confidence))`,
              [user.id, photoId, confidence]
            );
          }
        }
      }
    }
```

Frontend Component: src/Gallery/Gallery.jsx

```
import { useCallback, useEffect, useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { getStoredToken } from '../api/client';
import ImageViewer from '../components/ImageViewer';
import DirectionalTransition from '../components/DirectionalTransition';
import './Gallery.css';

const API_BASE = '/api';
const PAGE_LIMIT = 20;

const MATCH_STAGES = [
  { label: 'Analysing your selfie...', icon: '\uD83D\uDD0D' },
  { label: 'Generating face embedding...', icon: '\uD83E\uDDEC' },
  { label: 'Comparing with gallery...', icon: '\uD83D\uDDBC' },
  { label: 'Scoring similarity...', icon: '\uD83D\uDCCA' },
  { label: 'Finalising your matches...', icon: '\u2728' },
];

function RematchLoader() {
  const [stageIdx, setStageIdx] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setStageIdx((i) => (i + 1) % MATCH_STAGES.length);
    }, 2000);
    return () => clearInterval(id);
  }, []);

  const stage = MATCH_STAGES[stageIdx];
```

```
  return (
    <div className="rm-loader" role="status">
      <div className="rm-face-wrap">
        <div className="rm-scan-line" />
      </div>
      <p className="rm-stage-label">
        <span>{stage.icon}</span> {stage.label}
      </p>
    </div>
  );
}

export default function Gallery() {
  const { user } = useAuth();
  const [photos, setPhotos] = useState([]);
  const [page, setPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(true);
  const [selectedPhoto, setSelectedPhoto] = useState(null);

  const fetchGallery = useCallback(async (targetPage = 1) => {
    setLoading(true);
    try {
      const res = await fetch(`${API_BASE}/gallery?page=${targetPage}&limit=${PAGE_LIMIT}`, {
        headers: { Authorization: `Bearer ${getStoredToken()}` },
      });
      const data = await res.json();
      setPhotos(data.images || []);
      setTotal(data.total || 0);
      setTotalPages(data.totalPages || 1);
      setPage(data.page || 1);
    } catch (err) {
      console.error('Gallery fetch error:', err);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    fetchGallery(1);
  }, [fetchGallery]);
```

```
return (
  <DirectionalTransition>
    <div className="gal-page">
      <div className="gal-container">
        <header className="gal-header">
          <h1 className="gal-title">Your Personal Photo Gallery</h1>
          <p className="gal-subtitle">
            {total > 0
              ? `Showing ${total} photograph(s) matched using your registered facial profile`
              : 'Upload your reference selfie in settings to begin facial indexing'}
          </p>
        </header>

        {loading ? (
          <RematchLoader />
        ) : photos.length === 0 ? (
          <div className="gal-empty-state">
            <p>No matched photos found yet. Check back soon as event albums are uploaded.</p>
          </div>
        ) : (
          <div className="gal-grid">
            {photos.map((img) => (
              <div
                key={img.id}
                className="gal-card"
                onClick={() => setSelectedPhoto(img)}
              >
                <img
                  src={img.thumbnail_path || img.filepath}
                  alt={img.filename}
                  className="gal-card__img"
                  loading="lazy"
                />
                <div className="gal-card__badge">
                  {(Number(img.match_confidence) * 100).toFixed(0)}% Match
                </div>
              </div>
            ))}
          </div>
        )}

        {totalPages > 1 && (
```

```
          <div className="gal-pagination">
            <button
              disabled={page <= 1}
              onClick={() => fetchGallery(page - 1)}
              className="gal-page-btn"
            >
              Previous
            </button>
            <span>Page {page} of {totalPages}</span>
            <button
              disabled={page >= totalPages}
              onClick={() => fetchGallery(page + 1)}
              className="gal-page-btn"
            >
              Next
            </button>
          </div>
        )}

        {selectedPhoto && (
          <ImageViewer
            photo={selectedPhoto}
            onClose={() => setSelectedPhoto(null)}
          />
        )}
      </div>
    </div>
  </DirectionalTransition>
 );
}
```

Backend Controller: backend/controllers/galleryController.js (getUserGallery)

```
import pool from '../config/db.js';

export async function getUserGallery(req, res) {
  const limit  = Math.min(Math.max(parseInt(req.query.limit) || 20, 1), 100);
  const page   = Math.max(parseInt(req.query.page) || 1, 1);
  const offset = (page - 1) * limit;

  try {
    const [[{ total }]] = await pool.execute(
      `SELECT COUNT(*) AS total
```

```
        FROM user_gallery_photos ugp
        JOIN images i ON i.id = ugp.photo_id
        WHERE ugp.user_id = ?`,
       [req.user.id]
    );

    const [rows] = await pool.query(
      `SELECT
        i.id,
        i.filename,
        i.filepath,
        i.thumbnail_path,
        i.uploaded_at,
        ugp.match_confidence,
        ugp.matched_at
       FROM user_gallery_photos ugp
       JOIN images i ON i.id = ugp.photo_id
       WHERE ugp.user_id = ?
       ORDER BY ugp.matched_at DESC
       LIMIT ${limit} OFFSET ${offset}`,
      [req.user.id]
    );

    const totalCount = Number(total) || 0;
    const totalPages = Math.max(Math.ceil(totalCount / limit), 1);

    return res.json({
      images: rows,
      total: totalCount,
      page,
      totalPages,
      limit,
    });
  } catch (err) {
    console.error('[getUserGallery error]', err.message);
    return res.status(500).json({ message: 'Failed to fetch your gallery.' });
  }
}
```